

RESEARCH ARTICLE

An Edge-Deployed Real-Time Adaptive Traffic Light Control System Using YOLO-Based Vehicle Detection and PCE-Aware Density Estimation

MEHWISH RAZA¹, MAJIDA KAZMI^{1,2}, HAMZA MUNIR KIDWAI², HASHIM RAZA KHAN^{1,2},
SAAD AHMED QAZI^{1,2}, KAMRAN ARSHAD^{3,4}, (Senior Member, IEEE),
AND KHALED ASSALEH^{3,4}, (Senior Member, IEEE)

¹Faculty of Electrical and Computer Engineering, NED University of Engineering and Technology, Karachi 75270, Pakistan

²Neuro Computation Laboratory, National Centre of Artificial Intelligence, NED University of Engineering and Technology, Karachi 75270, Pakistan

³Department of Electrical and Computer Engineering, College of Engineering and Information Technology, Ajman University, Ajman, United Arab Emirates

⁴Artificial Intelligence Research Centre, Ajman University, Ajman, United Arab Emirates

Corresponding author: Majida Kazmi (majidakazmi@neduet.edu.pk)

This work was supported by Ajman University Internal Research under Grant 2024-IRG-ENIT-2.

ABSTRACT This paper presents an edge-deployable, real-time adaptive traffic light control system designed for complex and undisciplined urban traffic environments. The system integrates computer vision and Internet of Things (IoT) technologies to enable smart traffic management. A robust dataset was developed to capture diverse traffic conditions, including occlusions and unstructured vehicle movement. A lightweight vehicle detection model, based on (You Only Look Once) YOLO and deployed on a Jetson Xavier NX-powered roadside edge node (RSEN), enables real-time traffic monitoring with a mean Average Precision (mAP) of 90% and 74 frames per second (FPS). To improve traffic flow representation, we introduce a traffic density estimation method that incorporates Passenger Car Equivalent (PCE) factors, considering the varying impact of different vehicle types. The estimated density is used by an adaptive traffic light control (ATLC) algorithm, deployed on a resource-constrained Portenta H7 board, which dynamically adjusts signal timings using real-time data, starvation prevention, and tie-breaking mechanisms. The proposed system reduces traffic congestion at intersections by up to 33% and waiting times by 23% compared to traditional fixed-time controls. The proposed framework is cost-effective, scalable, and fully operational on edge devices, offering a practical solution for real-world smart city deployments.

INDEX TERMS Adaptive traffic signal control, deep learning, edge computer vision, hardware implementation, intelligent transportation systems, Internet of Things, real-time vehicle detection, YOLO, traffic density estimation.

I. INTRODUCTION

Intelligent Transportation Systems (ITS) integrate Internet of Things (IoT) technologies, including sensors, communication networks, and real-time data analytics, to enhance safety, efficiency, and sustainability in modern transportation systems [1]. By utilizing IoT-enabled devices such as connected vehicles and roadside sensors, ITS facilitates proactive traffic management, thereby reducing congestion,

accidents, and environmental pollution [2]. Over the past decade, ITS has gained significant attention for its role in optimizing energy consumption, enhancing road safety, and minimizing the environmental impact of transportation systems.

Traditional traffic management systems, particularly fixed-time traffic lights, operate on predetermined cycles without considering real-time traffic conditions [3]. This approach is inefficient, as traffic patterns can vary significantly due to factors such as peak hours, accidents, public events, and heterogeneous traffic flows. To address

The associate editor coordinating the review of this manuscript and approving it for publication was Adrian Stern¹.

TABLE 1. Comparison of sensor-based methods and computer vision-based techniques for traffic monitoring.

Feature	Inductive Loops [4]	Pressure Plates [5]	Ultrasonic Sensors [6]	Magnetometers [7]	Computer Vision (CV)-Based Techniques [8]
Detection Coverage	Fixed, limited to embedded loop area	Fixed, limited to plate area	Limited range, affected by obstructions	Localized, detects only in proximity	Wide-area coverage, monitors entire intersections and roads
Ability to Detect Sudden Traffic Changes	No, only detects vehicles passing over the loop	No, only detects direct contact	Limited, struggles with occluded or dense traffic	No, detects only passing vehicles	Yes, continuously tracks vehicle movements and congestion patterns
Vehicle Type Identification	No, detects only vehicle presence	No, cannot differentiate between vehicles	Limited, struggles with differentiation	No, detects only metallic objects	Yes, differentiates between vehicle types, pedestrians, and cyclists
Scalability	Low, requires invasive road modifications	Low, requires physical installation	Moderate, but limited range	Low, fixed at specific locations	High, deployable using existing camera infrastructure
Environmental Sensitivity	Affected by road damage and maintenance	Susceptible to mechanical wear	Affected by weather (rain, fog, temperature variations)	Affected by magnetic interference	Works under various lighting conditions with advanced image processing
Maintenance Cost	High, requires roadwork for installation and repair	High, prone to wear and tear	Moderate, but frequent calibration needed	Moderate, periodic recalibration needed	Low, software updates enhance performance without physical modifications
Additional Applications	Limited to traffic signal control	Vehicle counting only	Basic vehicle detection	Basic vehicle presence detection	Adaptive traffic control, traffic violation detection, accident monitoring, public safety surveillance, and pedestrian tracking
Integration with Smart City Initiatives	No, standalone detection	No, standalone detection	Limited, requires integration with other sensors	No, standalone detection	Yes, integrates with AI-based traffic management and urban safety systems

these limitations, adaptive traffic light systems have been developed that dynamically adjust signal timings based on real-time traffic data.

Real-time traffic data can be collected using two primary approaches: sensor-based methods and computer vision (CV)-based methods. Traditional sensor-based techniques, such as inductive loops [4], pressure plates [5], ultrasonic sensors [6], and magnetometers [7], rely on physical sensors embedded in or near roadways to detect vehicle presence and movement. However, these systems have several inherent limitations. First, they are confined to fixed locations, rendering them incapable of detecting congestion, accidents, or lane changes beyond their immediate vicinity. Second, they struggle to handle sudden changes in traffic flow, such as those caused by accidents or peak-hour congestion.

To overcome these shortcomings, CV-based techniques have emerged as a compelling alternative. CV systems leverage real-time image processing and deep learning models to provide continuous, wide-area monitoring without invasive road modifications. These systems enable real-time traffic flow analysis, multi-modal detection of different vehicle types and pedestrians, and adaptability to unstructured traffic conditions [8]. Beyond traffic scheduling, CV-based techniques also play critical roles in safe city initiatives, violation detection, accident response, and pedestrian safety monitoring. The growing availability of rich datasets, powerful computing platforms, and deep neural networks has further accelerated the adoption of CV-based ITS solutions.

A detailed comparison between sensor-based and CV-based approaches is presented in Table 1, highlighting the advantages of CV techniques in terms of detection coverage, scalability, and adaptability to dynamic traffic conditions.

CV-based traffic analysis uses normally two types of cameras: drone-based cameras and mounted cameras. Several studies have explored UAV-based traffic monitoring. For instance, [9] proposed a lightweight YOLOv8-based model for drone applications, achieving high speed (251 FPS) and accuracy (mAP: 86.2%) on the VisDrone2019 dataset. Similarly, [10] integrated YOLOv8 with drone platforms,

reporting 79% and 83% accuracy for YOLOv8-n and YOLOv8-x, respectively. Another study [11] focused on vehicle detection using drone based aerial imagery, balancing accuracy and computational efficiency. While drone-based systems provide a wide field of view, they pose inherent challenges such as regulatory restrictions [12], weather dependency, flight duration limitations [13], and the need for frequent battery recharge, making them less suitable for roadside traffic management in real-world deployment [14]. Since our primary objective is adaptive traffic light control, we focus on mounted camera feeds for traffic data collection, as they offer continuous real-time monitoring and seamless integration with traffic signal systems. Drone-based solutions fall outside the scope of this study due to their operational constraints and limited feasibility for real-time traffic signal adaptation.

Most existing CV-based traffic management solutions are designed for disciplined traffic, assuming well-defined lanes and predictable vehicle movements. However, these models struggle in undisciplined environments, where traffic is chaotic, lanes are poorly defined, and frequent occlusions and unpredictable movements make detection more complex. Since they are trained on disciplined traffic data, they fail to recognize these variations, leading to significant accuracy drops. The high degree of occlusion, irregular vehicle positioning, and lack of defined lanes further challenge conventional models, making them ineffective in undisciplined scenarios.

Additionally, most studies use public datasets containing common vehicle types, which fail to capture the diversity of local traffic compositions in regions like South Asia. As a result, existing models lack the capability for fine-grained vehicle classification, limiting their applicability in environments with motorcycles, rickshaws, and other region-specific vehicles. Addressing these challenges requires an adaptive approach capable of real-time detection, classification, and traffic regulation.

Moreover, most of the current CV studies in the traffic domain focus on extracting standalone modules (e.g., vehicle

TABLE 2. List of abbreviation.

Abbreviation	Full name	Abbreviation	Full name
ATLC	Adaptive Traffic Light Control	NAS	Neural Architecture Search
CBL	Convolution Batch Normalization Leaky ReLU	OSM	Open Street Map
CNN	Convolutional Neural Networks	PAN	Path Aggregation Networks
CV	Computer Vision	PCE	Passenger Car Equivalent
DL	Deep Learning	R-CNN	Region-based Convolutional Neural Network
E-ELAN	Extended Efficient Layer Aggregation Network	R-FCN	Region-based Fully Convolutional Network
FPS	Frames Per Second	RGB	Red Green Blue
GDC	Green Denial Counter		
GNN	Graph Neural Networks	RSEN	Road Side Edge Node
GPU	Graphics Processing Unit	SPP	Spatial Pyramid Pooling
HOG	Histogram of Oriented Gradient	SSD	Single Shot Detector
IAA	Inter-Annotator Agreement	SUMO	Simulation of Urban MObility
IoT	Internet of Things	TRaCI	Traffic Control Interface
ITS	Intelligent Transportation Systems	AAV	Autonomous Aerial Vehicle
mAP	mean Average Precision	YOLO	You Only Look Once
MP	MaxPooling		

counts, vehicle types, and traffic densities), but the integration of these extracted parameters into a scheduler module for real-time traffic management is largely missing. To address this limitation, our proposed work not only extracts meaningful information from traffic frames but also utilizes this data to design a decision-making mechanism for adaptive traffic light control. Further, our work differs from previous studies by focusing on undisciplined traffic scenarios, which are often overlooked despite being prevalent in many developing regions. By integrating CV-based techniques with adaptive traffic light management, we enable real-time decision-making tailored to dynamic road conditions. Unlike cloud-based solutions that introduce delays, our approach leverages edge-based processing for low-latency responsiveness. Additionally, we incorporate Passenger Car Equivalent (PCE) factors for a more accurate representation of traffic density, rather than treating all vehicles equally. Finally, our system is fully implemented on hardware, demonstrating its real-world feasibility, unlike many studies that remain limited to simulations or theoretical models. The key contributions of this work are as follows:

1) **Development of a Contextually Diverse Dataset for Undisciplined and Complex Traffic Environments:**

A comprehensive data pipeline is developed, including real-world traffic data collection, pre-processing, precise manual annotation, and targeted contextual augmentations. The augmentation pipeline accounts for temporal, weather, behavioral, and camera-related variability to enhance model generalization.

2) **Traffic Density Estimation with PCE Factors:**

We introduced an efficient approach for traffic density estimation that incorporates Passenger Car Equivalent (PCE) factors based on vehicle size and shape. Unlike traditional methods that treat all vehicles equally, our approach accounts for the varying impact of different vehicle types, providing a more accurate representation of traffic dynamics. This detailed

understanding of traffic flow enables better optimization and decision-making in adaptive traffic management systems.

3) **Road-Side Edge Node (RSEN) Design for Real-Time Detection and Density Estimation:**

A dedicated RSEN was implemented on Jetson Xavier NX, featuring a lightweight, edge-optimized vehicle detection and traffic density estimation module, capable of on-device processing real-time video streams, with minimal latency, enabling on-the-fly responsiveness and infrastructure-independent deployment, essential for dynamic traffic conditions.

4) **Edge-Deployed Adaptive Traffic Light Control (ATLC) Algorithm:**

We propose an ATLC algorithm designed to dynamically adjust signal timings based on real-time traffic density and lane functionality. The algorithm incorporates starvation prevention and tie-breaking mechanisms to handle complex traffic scenarios effectively. It is deployed on the resource-constrained H7 Portenta board, demonstrating its suitability for edge-level traffic control in complex and undisciplined traffic environments.

5) **End-to-End Edge-Deployed System for Adaptive Traffic Management:**

A fully integrated, end-to-end adaptive traffic management system is deployed entirely on edge devices, combining vehicle detection, traffic density estimation, and signal control. The system is optimized for real-time operation and practical deployment in chaotic, undisciplined traffic environments.

6) **Comprehensive Evaluation of System's Generalizability and Effectiveness:**

Extensive evaluation is conducted to assess the generalizability of the vehicle detection module under varying times of day, weather conditions, and geographically diverse intersections. Additionally, the ATLC algorithm

TABLE 3. Literature review on recent cv-based techniques for traffic parameter computations.

Traffic	Reference, Year	Edge Deployment	Vehicle Detection	Road Occupancy	Traffic light Scheduling	Other	Dataset
Disciplined	[40], 2025	✓	✓	×	×	License plate recognition	Roboflow Datasets
	[33], 2023	×	✓	×	×	—	KITTI
	[77], 2023	×	✓	×	×	—	COCO, PKU, DAWN
	[38], 2024	×	✓	×	×	—	COCO
	[76], 2024	×	✓	×	×	—	Dawn COCO
	[74], 2024	×	✓	×	×	Vehicle counting	COCO BDD100K
	[75], 2025	×	✓	×	×	—	KITTI
	[29], 2022	×	✓	×	×	—	Citiscapes
	[30], 2022	✓	✓	×	×	—	KITTI
	[31], 2023	×	✓	×	×	—	COCO 2017
	[32], 2022	×	✓	×	×	—	Argoverse-HD
	[35], 2022	×	✓	✓	×	—	DS-ivic (Peru)
	[36], 2023	×	✓	×	×	—	MS COCO
							Argoverse-HD
Undisciplined	[18], 2023	×	✓	×	×	—	FLIR thermal, FLIR RGB, MB7500, and KITTI
	[34], 2021	×	✓	×	×	—	KITTI
	[37], 2023	✓	✓	×	×	—	DTLD
	[39], 2021	✓	✓	×	×	—	DHAKA AI
							IITM HETRA

is evaluated through simulation to verify its effectiveness in reducing congestion and preventing starvation.

The rest of this paper is structured as follows. Section II reviews related work. Section III details the proposed method. Section IV presents a comprehensive evaluation of the proposed framework. Section V presents a comparative analysis of recent methodologies for adaptive traffic signal control. Section VI outlines the limitations of the current work and suggests directions for future research. Finally, Section VII brings the paper to a conclusion. The abbreviations in this paper are listed in Table 2.

II. RELATED WORK

A. CV BASED TECHNIQUES FOR TRAFFIC PARAMETER COMPUTATIONS

Due to the rapid increase in vehicles on the road, conventional traffic systems are becoming inefficient, leading to longer waiting times, higher accident rates, pollution, and congestion. As a result, ATLC systems with real-time vehicle detection become inevitable [12]. Traditional image processing techniques, such as Background Subtraction [15], Histogram of Oriented Gradients (HOG) [16], and Haar features [17], rely on manual feature extraction and struggle with occlusion, traffic-less scenarios, and variations in background, illumination, and noise [18]. In contrast, Deep Learning (DL) has revolutionized object detection and image classification, enabling automatic feature extraction through data-driven algorithms. DL-based methods, particularly Convolutional Neural Networks (CNNs), have become the backbone of modern vehicle detection systems [19]. These methods are broadly categorized into two-stage and one-stage detection models. Two-stage models, such as Faster R-CNN [20], Mask

R-CNN [21], and R-FCN [22], first generate regions of interest and then classify and refine object locations. One-stage models, such as Single Shot Detector (SSD) [23] and You Only Look Once (YOLO) [24], [25], [26], [27], [28], predict bounding boxes and class probabilities in a single pass, offering faster inference speeds but sometimes at the cost of accuracy in dense or small-object scenarios [18].

Table 3 summarizes the recent advancements in CV-based models and their respective contributions in ITS. For instance, [29] enhanced YOLOv4 by incorporating channel attention and a high-resolution module, achieving higher accuracy but at the cost of increased model size (over 250 MB), making it unsuitable for edge deployment. To address this, [30] introduced Edge YOLO, a lightweight version of YOLOv4 with a reduced size of 25 MB, enabling faster inference speeds. However, the proposed solution is designed for modern vehicles and lacks adaptability for undisciplined traffic scenarios, limiting its applicability in regions with diverse and unpredictable traffic conditions. Similarly, [31] applied YOLOv7 for vehicle detection using the Argoverse-HD and DS-ivic datasets, its deployment on high-power computing platforms (8-core processors) and low accuracy (57%) make it impractical for real-world edge applications. In another study, [32] proposed a modified CondenseNeXt model for vehicle detection, achieving 35% (Argoverse) and 58% (MsCOCO) accuracy on the NVIDIA Tesla V100 platform. Due to its high computational power and low accuracy its practicality for edge deployment is limited. In another study, [33] applied ResNet50 for vehicle detection on the KITTI dataset, achieving 82.12% accuracy. However, the model's lack of edge deployment and focus on disciplined traffic scenarios limit its practicality. Reference [34] presented a

YOLOv3-based model for real-time vehicle detection in undisciplined traffic using the DhakaAI dataset. Although the model is fast, it struggles with detecting obstructed vehicles and achieves lower accuracy.

Due to low accuracy reported in previous studies, recent research has explored hybrid approaches, such as the ensemble model proposed in [35], combine Faster R-CNN and YOLO to achieve high accuracy. However, the computational complexity of the ensemble framework results in slower inference speeds, making it unsuitable for real-time deployment. On the other hand, [36] introduced an enhanced YOLOv5 model with structural re-parametrization and neural architecture search, achieving notable accuracy and speed. However, the model does not address the challenges of undisciplined traffic and is implemented on high-power GPUs (NVIDIA GeForce RTX), which are unsuitable for edge deployment due to their power consumption and size. Similarly, in another approach, [37] fused ResNet and DenseNet for vehicle detection, achieving suboptimal accuracy and low FPS on the DTLT and KITTI datasets. The large size of the ensemble model further limits its applicability in edge computing environments.

In addition to the hybrid approaches recently a few studies have also explored transformer-based architectures for vehicle detection. For example, [18] proposed a Swin Transformer backbone with a Mask R-CNN head for detecting vehicles in undisciplined traffic scenarios. While the model achieves commendable accuracy, its inference speed and high power consumption (due to GPU deployment) make it unsuitable for real-time edge applications. Similarly, [38] proposed a detection transformer DETR-SPP, another transformer-based model for vehicle detection, achieving 51.31% accuracy on the COCO dataset. The model's lower accuracy and lack of edge deployment reduce its suitability for real-time ITS applications.

While many previous studies focused on improving accuracy and computational efficiency, they often lacked feasibility for edge deployment due to high model sizes, power consumption, or reliance on high-performance computing platforms. Recognizing these limitations, recent research has shifted towards developing lightweight and resource-efficient models suitable for real-time edge applications. In [39] proposed DhakaNet, a low-resource DL architecture incorporating Cross-Stage Partial Networks and Path Aggregation Networks. While the model achieves 95% accuracy on the IITM Hetra dataset, its performance on the Dhaka AI dataset is significantly lower (9.9%). Additionally, the model's deployment on a Raspberry Pi highlights its potential for edge applications but also reveals limitations in accuracy and speed. The authors in [40] proposed a self-adaptive traffic signal system using YOLOv8 and YOLO11 implemented on RaspberryPi, achieving 95% accuracy on the Roboflow dataset. However, the system is designed for disciplined traffic and lacks real-world deployment, limiting its applicability in chaotic environments.

B. ADAPTIVE TRAFFIC LIGHT CONTROL

The traffic parameters derived from the above computer vision-based studies can be utilized for developing ATLC. In this regard, numerous research studies have been presented, which can be categorized into learning-based and rule-based approaches.

The learning-based approach primarily consists of Reinforcement Learning (RL) techniques [41], where traffic signal control is modeled as a sequential decision-making problem. An RL agent (traffic signal controller) interacts with the environment (traffic conditions) to optimize signal timing. Traffic conditions are encoded into a state representation, guiding the agent in selecting actions such as adjusting signal durations. A reward function, based on metrics like congestion reduction and minimized wait times, drives learning. Through repeated interactions, algorithms like DQN (Deep Q Networks) [42] optimize the traffic scheduling policy.

Several studies employ reinforcement learning (RL) for traffic signal control. Reference [43] introduces a multi-agent RL approach with graph neural networks (GNNs) for traffic signal management, demonstrating an average travel time of 47.95 seconds in a CityFlow simulation. However, its effectiveness in real-world complex traffic scenarios remains unverified. Similarly, [44] presents an improved deep RL approach that enhances state and reward consistency, achieving a travel time of 145.2 seconds in SUMO. Despite its performance improvements, this approach has not been deployed in practical settings. [45] optimizes traffic light phase durations based on past traffic patterns using deep RL, reporting a 25% reduction in waiting time compared to fixed-time signals. However, despite their promising results, these RL-based methods remain confined to simulation environments due to their high computational demands, making real-world deployment challenging. While RL techniques offer adaptability, they require extensive training, large datasets, and high processing power, making real-time deployment infeasible on edge devices with limited resources [46]. Most RL-based traffic scheduling approaches rely on cloud-based processing, introducing latency and network dependency, which hinders real-time adaptive scheduling. As a result, these solutions remain largely simulation-based, with minimal practical deployment in real-world traffic systems. This limitation necessitates exploring alternative approaches that are computationally efficient and feasible for real-world implementation.

Unlike learning-based methods, rule-based approaches rely on predefined algorithms and rule-based logic to adjust traffic signals in real-time based on traffic parameters. These models are typically more interpretable and computationally efficient, making them ideal for real-time applications in resource-constrained environments. In [47], the authors propose a framework that integrates adaptive traffic lights with emergency vehicle communication, prioritizing emergency vehicles at intersections and adjusting signals based on

vehicle counts from traffic video. However, the system does not account for variations in vehicle sizes and shapes and remains simulation-based without edge deployment. Another rule-based traffic signal control is presented in [48]. The authors have proposed fuzzy logic-based signal controller to manage traffic at isolated intersections. The system dynamically adjusts the traffic signal cycle length based on the real-time flow rates detected by the on-road sensors. While the system is less complex but it not accurate as it does not differentiate between vehicle types (e.g., cars, buses, trucks), which can have different impacts on traffic flow and signal timing. Moreover, the system's performance is validated through simulation which may not fully replicate real-world traffic conditions. The lack of real-world deployment or edge implementation limits the generalizability of the results.

Despite these advancements, several research gaps persist in the existing literature:

- Most vehicle detection frameworks are designed for controlled, disciplined traffic scenarios and fail to address the challenges of undisciplined, occluded, and dense traffic, which are common in South Asian countries.
- The majority of studies rely on public datasets (e.g., COCO, KITTI) that primarily include standard vehicle types such as cars and trucks. However, these datasets lack representation of diverse vehicle classes such as auto rickshaws, motorcycles, and animal-drawn carts, which are prevalent in many real-world traffic scenarios.
- Many existing models depend on high-performance GPU for inference, making them unsuitable for edge deployment due to high power consumption and resource constraints.
- Most adaptive traffic light frameworks are evaluated in simulations rather than real-world deployments, limiting their practical applicability.

These gaps highlight the need for a comprehensive solution that combines real-time vehicle detection, traffic density estimation, and adaptive signal control in resource-constrained environments. Our work addresses these challenges by proposing a lightweight, edge-deployable framework that integrates real-time vehicle detection with adaptive traffic light control, demonstrating significant improvements in traffic flow optimization and congestion reduction.

III. PROPOSED METHODOLOGY

The distributed primary-secondary architecture as shown in Fig. 1 performs efficient processing and decision-making while maintaining real-time responsiveness in urban traffic management scenarios. The proposed framework consists of three significant components, as shown in Fig. 1: the first component involves gathering, preparing, and enhancing data, while the second component comprises RSEN that is responsible for detecting and categorizing vehicles and computing traffic density in this phase. The third component, the ATLC unit handles traffic scheduling. The two main

blocks of the framework, RSEN and ATLC nodes are further implemented on target edge devices to achieve real-time processing at a high frame rate.

Each RSEN node acting as a secondary node is responsible for local vehicle detection and traffic density estimation. It communicates its findings to the ATLC scheduling node which serves as the central primary node, as shown in Fig. 1. The distributed primary-secondary architecture performs efficient processing and decision-making while maintaining real-time responsiveness in urban traffic management scenarios.

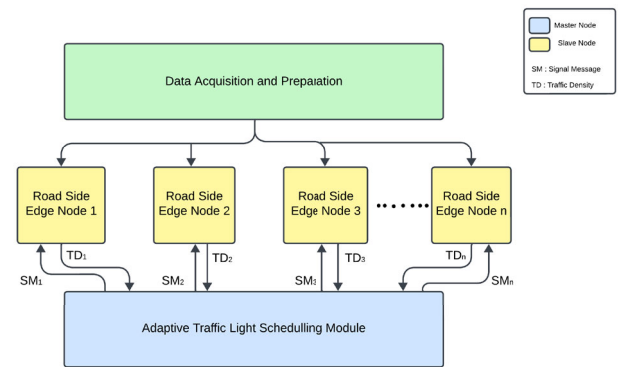


FIGURE 1. Workflow of the proposed framework.

A. DATA ACQUISITION AND PREPARATION

The process of data acquisition and preparation involves several steps, including data collection, preparation, annotation, splitting into training and testing samples, and augmentation. These steps are illustrated in Fig. 2.

1) DATA COLLECTION

Building a robust and diverse dataset is crucial for training a high-performing vehicle detection model. Given the lack of comprehensive datasets specific to South Asian regions, we developed an extensive dataset focused on the bustling traffic environment of Karachi, Pakistan. We collected traffic videos from Allama Shabbir Usmani road of Karachi to get clear frontal views of vehicles during the day. The diversity of real-world traffic scenarios, encompassing varying weather conditions and camera-induced artifacts, often introduces noise and degradation into the collected video streams. Ensuring high image quality and addressing these challenges, as explored in [49], is therefore critical for robust downstream processing and achieving reliable vehicle detection performance. Consequently, the raw video footage underwent a rigorous preparation pipeline, including targeted preprocessing and precise annotation, to mitigate these challenges and create a high-quality dataset for effective model training.

2) DATA PREPARATION

Videos generally adhere to SECAM standards, which have a standard frame rate of 25 FPS. Therefore, the video dataset used in this study also follows the standard 25 FPS frame rate,

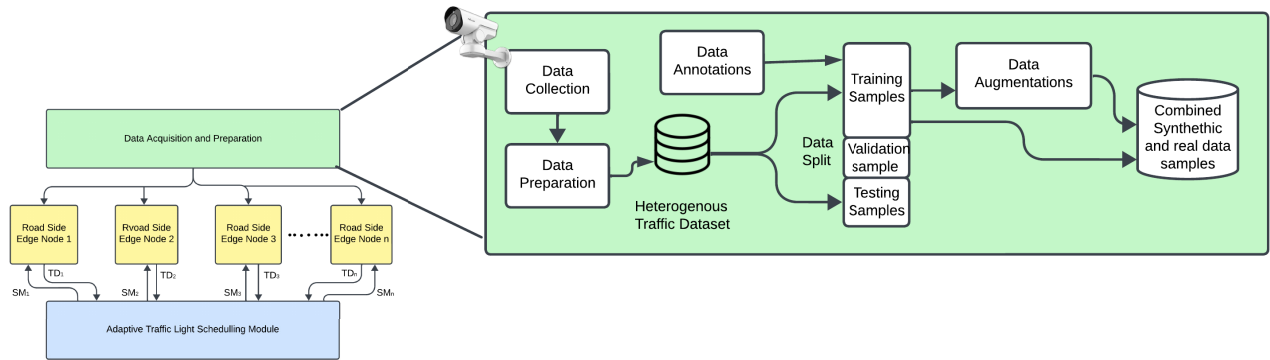


FIGURE 2. Data acquisition and preparation.

and the images in the dataset have a resolution of 1280×738 pixels. The complexity of the data set was classified as medium to high due to the presence of occlusions and vehicles overlapping. The high vehicle density and lack of lane discipline often resulted in vehicles being overshadowed by those directly behind them. To optimize storage and processing while capturing traffic dynamics, we employed an adaptive frame extraction technique at a 25% recording ratio, capturing one image every four seconds. This approach balances data capture with redundancy reduction. To ensure a fair evaluation of generalization, a strict 80:10:10 non-overlapping split was used, where augmentation was applied only to the training set, while validation and test sets were kept clean, unaugmented, and entirely unseen, as shown in Fig. 2. The training samples are then annotated to proceed with model training.

3) DATA ANNOTATION

Recent studies have shown that annotation quality plays an important role in the accuracy of detection models, particularly in complex real-world traffic scenes involving occlusions and overlapping vehicles, scenarios that are common in the chaotic and heterogeneous traffic environments of Pakistan. Poor and inconsistent annotations often lead to significant performance degradation even in advanced algorithms [50]. To ensure high-quality and consistent annotations under such conditions, we adopted a multi-layered approach, comprising clear annotation rules, expert arbitration, and routine quality checks throughout the dataset [51].

In this work, we annotated the training samples of the comprehensive dataset, capturing the diverse vehicular landscape of Karachi. However, a significant challenge during annotation was dealing with complex scenes involving partial occlusions and overlapping vehicles, common in densely packed, heterogeneous traffic. To ensure annotation consistency and accuracy under such conditions, we adopted a structured, multi-stage annotation protocol with the following components:

- **Category Guidelines:** We first developed detailed annotation guidelines that clearly defined all vehicle classes, supported by visual references. Annotators were trained to handle occluded vehicles by labeling only the visible parts while inferring the class using context clues like shape, size, and visible features.
- **Polygon-based Annotations:** For scenes with high vehicle overlap, we used polygon-based annotation tools instead of standard bounding boxes. This allowed annotators to more precisely outline vehicle boundaries, especially in crowded or partially visible scenarios.
- **Dual Annotation with Arbitration:** Each image, especially those with occlusions, was independently annotated by two annotators. In cases of discrepancy, a senior annotator performed a review and made the final decision to ensure the consistency of the label.
- **Inter-Annotator Agreement (IAA) Checks:** Regular IAA checks were conducted to evaluate consistency across annotators. In case of disagreement, a senior annotator reviewed the case and provided a final verdict to maintain label uniformity.
- **Routine Quality Control:** A dedicated quality control team periodically reviewed the annotated samples, with a special focus on complex cases (overlapping and occluded), to detect inconsistencies or systematic bias.

These measures ensured the high quality of our annotated dataset, enabling our proposed vehicle detection model to learn the distinguishing features of each category even in challenging scenarios. The precise and careful annotation enabled the model to generalize its knowledge and achieve superior performance in real-world conditions. Table 4 provides a summary of the dataset statistics, including the number of source images, annotations, class count, and class names.

4) COMPARISON WITH EXISTING DATASETS

To highlight the uniqueness and comprehensiveness of our dataset, we compare it with other publicly available datasets

TABLE 4. Summary of dataset before augmentation.

Category	Count
Source Images	1373
Class count	8
Class Names	Auto Rickshaw, Car-Jeeps-vans-Taxi Cart-Chingchi, Large Buses, Mini Buses Motorcycles-Scooters, Pickups, Trucks and Trailers

used for vehicle detection. The comparison is summarized in Table 5, showcasing key attributes such as image resolution, training/test split, source of data collection, diversity of vehicles, challenge and difficulty level, occlusion, and heavy chaotic traffic.

This comparison highlights the strengths of our dataset, particularly its focus on the unique traffic characteristics of South Asian urban environments, which include high vehicle density, diverse vehicle types, and frequent occlusions.

5) DATA AUGMENTATION

Data augmentation has become essential in modern computer vision, enabling deep convolutional neural networks to achieve impressive results by improving their ability to generalize to unseen data. Recent studies [63] highlight that carefully designed augmentation pipelines, rather than overly aggressive and random transformations, are more effective, as realistic variations better mimic real-world conditions and lead to higher accuracy. The current trend emphasizes strategic combinations of augmentations [64] that reflect practical scenarios rather than applying arbitrary transformations.

Building on this insight, we adopted a context based augmentation strategy to enhance the generalization and simulate common visual distortions encountered in roadside camera footage. The basic augmentations applied are categorized as follows:

- Geometric alterations: Images underwent random cropping and resizing within a 0-20% zoom range, followed by rotations of up to $\pm 15^\circ$ horizontally and vertically, as illustrated in Fig. 3n and Fig. 3o.
- Color variations: Color transformations were used, including sepia overlays to mimic dusty or sandy environments and bluish tints to reflect foggy or cold-weather scenarios, as illustrated in Fig. 3c and Fig. 3d.
- Image parameter adjustments: Brightness and exposure were randomly perturbed within a $\pm 25\%$ range, representing the lighting variations of the real world, as illustrated in Fig. 3f and Fig. 3g.
- Spatial context augmentation: A cut-out technique removed three distinct 33% boxes from selected images, representing partial occlusions and complex traffic scenarios as shown in Fig. 3l.
- Subtle image distortions: Blur effects with a 1.5-pixel radius and noise injections up to 5% of pixels were incorporated to simulate sensor noise and image imperfections, as shown in Fig. 3j.
- Mosaic augmentation: For further diversity, we implemented a mosaic augmentation technique, stitching

together multiple images to create novel and challenging training examples, as shown in Fig. 3k.

Importantly, these augmentations were not applied separately but combined in meaningful ways to represent complex scenarios. We carefully selected and tuned combinations that would realistically reflect the challenges of real-world traffic. For example:

- A slightly rotated, blurred image with mild noise reflects a real traffic situation, like a moving vehicle caught by a camera with motion blur on a bumpy road.
- The combination of blur effect with cut-out was used specifically, to expose the model to realistic complexities, such as a vehicle that is both partially occluded (represented by cut-out) and simultaneously affected by blurring due to motion, camera focus issues, or adverse weather conditions (represented by blur). This helped the model to develop a stronger capacity for detecting objects even when their visibility is low.

This combined augmentation strategy served as a foundation for further contextual transformations, targeting weather conditions and contextual variability.

Context Based Augmentation: To further align the augmentation pipeline with real-world deployment conditions, a focused set of augmentation strategies was developed to represent weather-related challenges and contextual diversity, as illustrated in Fig. 3. These include synthetic overlays and filters that mimic rain streaks (Fig. 3a), snow textures (Fig. 3b), fog layers (Fig. 3c), and airborne dust (Fig. 3d). Adjustments in contrast, hue, tint, and brightness were incorporated to represent both atmospheric, temporal shifts, to reflect dusk (Fig. 3g) and night-time (Fig. 3h). Contextual factors such as complex traffic behaviors (e.g., occlusions, moving vehicles) and varying camera perspectives were also considered. The augmentation strategies were organized into four real-world context categories: *weather conditions*, *time-of-day variation*, *traffic behavior*, and *camera perspective*. A detailed mapping of these contexts to the applied augmentation techniques is provided in Table 6.

This context based augmentation strategy not only enhances visual diversity, but also improves the robustness of the model under operational uncertainty. After applying augmentation, the dataset comprised a total of 2,726 images. Of these, 80% (2,182 images) were used for training, while 10% (272 images) were allocated to validation and the remaining 10% (272 images) to testing.

B. ROAD SIDE EDGE NODES

Following the creation of a robust dataset, the next step is vehicle detection and classification. The RSEN is specifically designed to detect vehicles and compute traffic density in real-time, as shown in Fig. 4.

1) VEHICLE DETECTION MODEL

Real-world driving scenarios present a multitude of obstacles for accurate vehicle identification, including occlusions,

TABLE 5. Comparison of our dataset with existing benchmark datasets based on key attributes.

Feature	KITTI [53]	BDD100K [56]	Indian Driving Dataset [57]	Stanford [58]	Prince Sultan University [59]	Beijing Institute of Technology [60]	CompCars [61]	Dawn [62]	Our Dataset
Number of images	15,000	120,000,000	46,588	8,506	270	9,850	186,000	1,000	2,400
Image resolution	1242×375	1280×720	1920×1080	1409×1916	1920×1080	1920×1080	2100×1395	1920×1080	1280×738
Vehicle Classes	Car, Van, Truck, Tram	Bike (10,229), car (1,021,857), bus (16,505), truck (42,963), motor (4,296), train (179)	Bicycle, bus, train, motorcycle, car, truck, auto-rickshaw, caravan, animal, trailer	Bus (100), golfcart (200), car (1300), skateboard (300)	-	Sedan (5,922), SUV (1,392), microbus (883), truck (822), bus (558), minivan (476)	MPV, SUV, hatchback, sedan, minibus, fastback, estate, pickup, sports, crossover, convertible, hardtop convertible	Car (82.21%), bus (2.05%), truck (8.22%), motorcycles + bicycles (1.36%)	Auto Rickshaw, Car-Jeeps-vans-Taxi, Cart-Chingchi, Large Buses, Mini Buses, Motorcycles-Scooters, Pickups, Trucks and Trailers
Training/test split	80/20	70/30	67/33	80/20	80/20	80/20	-	-	80/20
Source of data collection	Vehicle on-board camera	Vehicle on-board camera	Vehicle on-board camera	3DR SOLO quadcopter	3DR SOLO drone and online source	Roadside fixed camera	Web images and surveillance camera	Web images	Roadside fixed camera
Diversity of vehicles	Limited variety	Wide variety	Wide variety	Wide variety	Limited variety	Limited variety	Wide variety	Limited variety	Wide variety
Occlusion	No	Yes	Yes	No	No	No	No	No	Yes
Heavy Chaotic Traffic	No	No	Yes	No	No	No	No	No	Yes

TABLE 6. Context based data augmentation strategy.

Type	Context	Applied Augmentation Techniques
Weather	Rainy, snowy, sandy/dusty, foggy	Rain: Semi-transparent rain streak overlays (random angle) with motion blur and brightness reduction. Snow: Snowflake textures, brightness adjustment, and white overlay on horizontal surfaces. Sand/Dust: Sepia tint, haze overlays, particle-like noise, and reduced sharpness. Fog: Depth-aware Gaussian haze, contrast reduction, and bluish tint.
Time of Day	Daylight, glare, afternoon, night-time visibility	- Brightness and contrast adjustment with histogram equalization. - Color desaturation and blue hue shift. - Shadow overlays and synthetic headlight glare.
Traffic Behavior	Occlusions, moving vehicles	- Cutout augmentation. - shear transformations. - Linear and motion blur to simulate dynamic motion.
Camera Perspective	Low-angle, side-view, varying zoom	- Affine transformation. - Tilt and in-plane rotation.

varying weather conditions, and motion blur. To navigate these complexities effectively, sophisticated deep learning models are deployed.

This study focuses on the YOLO algorithm due to its exceptional speed and single-pass object identification

capabilities. These features are particularly advantageous for edge real time deployment, where high speed is paramount. The YOLO architecture consists of three main parts: the backbone, neck, and head. Features are extracted by the backbone, fine-tuned by the neck, and predicted by the head for bounding boxes and class probabilities. The architecture of the YOLO model is illustrated in Fig. 5.

We conducted a comprehensive evaluation of several state-of-the-art object detection models, ranging from lightweight architectures like YOLOv7-tiny to various configurations of YOLOv8, including the nano and small variants. Additionally, we investigated the performance of YOLO-NAS, a recent advancement that leverages Neural Architecture Search (NAS), to identify the optimal trade-off between detection accuracy, model size, and real-time inference speed for our specific application in vehicle detection. This comparative study, performed on a carefully curated and annotated dataset, aims to determine the most effective model for accurate, efficient, and deployable traffic perception in resource-constrained edge environments.

YOLOv7 [27] represents a recent evolution in the YOLO family and introduces the Extended Efficient Layer Aggregation Network (E-ELAN), which enhances the learning and convergence capabilities of the network by effectively managing both short and long gradient paths [28], [65]. In our work, we specifically utilized the YOLOv7-tiny variant, which adopts Leaky-ReLU activations in place of the standard sigmoid functions used in the full YOLOv7 model. This architectural simplification reduces computational complexity while preserving competitive accuracy, making it well-suited for deployment on embedded systems.

In addition to YOLOv7-tiny, we evaluated multiple variants of YOLOv8, the latest model in the YOLO series developed by Ultralytics and released in 2023 [66]. YOLOv8

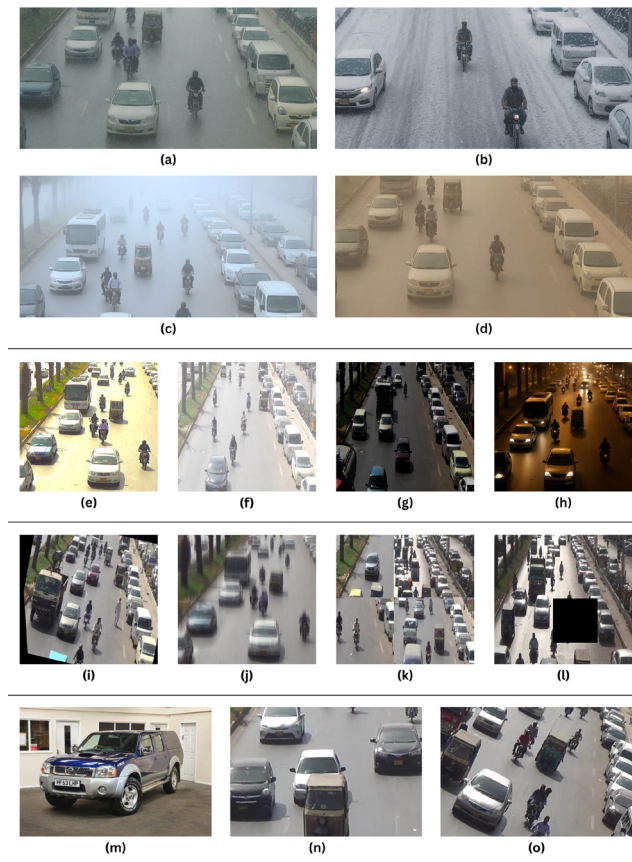


FIGURE 3. Consolidated visualization of context based data augmentation strategies designed to improve model generalization. (a–d) Weather augmentations: (a) rain (b) snow (c) fog (d) dust; (e–h) Time-of-day augmentations: (e) daylight (f) afternoon glare (g) dusk/evening (h) nighttime with headlight glare; (i–l) Traffic behavior: (i) shear (j) motion blur (k–l) occlusion; (m–o) Camera perspective changes: (m) side view, (n) zoom-in, (o) tilt/in-plane rotation.

introduces significant improvements over its predecessors (e.g., YOLOv5 and YOLOv7), including enhanced backbone design, anchor-free detection, and improved training strategies. The nano and small versions, in particular, feature minimal network depth and parameter count, resulting in reduced memory footprint and faster inference, which are the critical factors for real-time edge deployment.

Finally, we explored the potential of YOLO-NAS [35], [67], a model generated through Neural Architecture Search, which has not yet been extensively applied to the vehicle detection domain. In this study, we incorporate YOLO-NAS to assess its performance in Intelligent Transportation Systems (ITS) applications, representing one of the first such evaluations in this context.

The successful implementation of our vehicle detection system is highly dependent on the efficient training of the selected YOLO models. The training process involves optimizing the model parameters to accurately detect and categorize vehicles in various traffic situations.

During the training phase, an extensive investigation of the hyperparameters was carried out to refine the YOLO models.

We conducted a comprehensive grid search [68] to systematically explore the hyperparameter space for each model (YOLOv7-tiny, YOLOv8-nano, YOLOv8-small, and YOLO-NAS-small) and identify the optimal configurations for each model. The following hyperparameters were systematically varied:

- Learning Rate: [0.001, 0.01, 0.1]
- Batch Size: [16, 32, 64]
- Number of Epochs: [50, 100, 150]
- Optimizer: [SGD, Adam, AdamW]

The tuned hyperparameters, including learning rate, batch size, number of epochs, and optimizer type, were selected based on performance evaluation using mean Average Precision (mAP) as the primary metric. The specific hyperparameter values used in the training are summarized in Table 7.

TABLE 7. Hyperparameter tuning results for YOLO models.

Model	Learning Rate	Batch Size	Epochs	Optimizer
YOLOv7-tiny	0.01	32	100	SGD
YOLOv8-nano	0.001	16	100	AdamW
YOLOv8-small	0.01	32	100	AdamW
YOLO-NAS-small	0.005	32	50	AdamW

2) TRAFFIC DENSITY ESTIMATION

Following the vehicle detection model, the next important step in our framework is to accurately estimate the traffic density. Recognizing the unique size and shape of each detected vehicle is crucial for a deeper understanding of traffic flow dynamics, enabling the implementation of effective management strategies.

- **Impact of Vehicle Characteristics on Traffic Flow:** Regardless of its brand, every vehicle has distinct size and shape characteristics that influence its effect on traffic flow. To account for this variability, the proposed framework goes beyond a simple vehicle count generated by the vehicle detection model. Using the detected vehicle type, it assigns a corresponding **Passenger Car Equivalent (PCE) factor**. The assignment of PCE factors is based on a comprehensive analysis of vehicle size and its corresponding impact on traffic flow. These factors are carefully calibrated to accurately represent the influence of each vehicle on overall traffic density. Table 8 illustrates the PCE factors assigned for different vehicle sizes, providing a standardized framework for the estimation of traffic density.
- **Calculating Traffic Density with PCE Factors:**
 - **Foundation:** The vehicle count generated by the detection model serves as the starting point.
 - **Weighting by PCE:** For each detected vehicle type, its count is multiplied by its associated PCE factor. The PCE factors reported from other studies [13] of Karachi are used in this research.
 - **Cumulative Impact:** The individual weighted counts for all vehicle types are then summed, resulting in a single value reflecting the combined impact of all

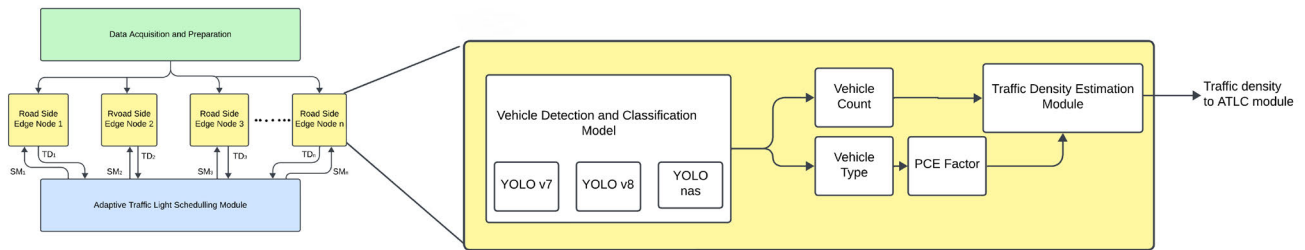


FIGURE 4. Road Side Edge Node.

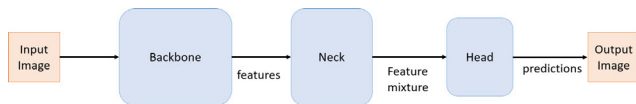


FIGURE 5. General YOLO Architecture.

vehicles on the road, as shown in the equation 1. This comprehensive value provides a clearer and more precise picture of the traffic situation.

$$\text{Traffic Density} = \sum_{i=1}^n (\text{Vehicle Count}_i \times \text{PCE}_i) \quad (1)$$

where: n is the total number of vehicle types considered, Vehicle Count_i denotes the number of vehicles of type i , PCE_i is the Passenger Car Equivalent (PCE) factor associated with vehicle type i .

TABLE 8. Vehicle classes and PCE factors.

Vehicle Classes	PCE Factor [13]
Cars/Jeeps/Vans/Taxi	1.00
Motorcycles/Scooters	0.25
Auto Rickshaw	0.50
Pickups	1.50
MiniBuses	2.00
Large Buses	2.50
Trucks & Trailers	3.00

In the Fig. 6, the traffic density is calculated and displayed on the top left side of the figure. This visualization aids in interpreting traffic conditions and facilitates informed decision-making in traffic management.

By precisely calculating the density of traffic, the framework acquires significant knowledge regarding the current traffic conditions. This valuable information serves as the input for the traffic scheduling component that utilizes these traffic density data to dynamically modify traffic flow, optimize lanes utilization, and implement measures to control congestion.

C. ADAPTIVE TRAFFIC LIGHT CONTROL MODULE

After estimating traffic density, each RSEN (secondary node) sends its respective traffic density to the ATLC module (primary node) via Wi-Fi. The ATLC module, using



FIGURE 6. Simulation of traffic density of the proposed framework.

Algorithm 1, dynamically adjusts signal timings based on real-time traffic density to ensure smooth flow and prevent starvation.

In the context of our study, starvation refers to a scenario where a particular signal phase is repeatedly denied the green signal across multiple cycles, despite the presence of waiting vehicles. To ensure fairness the system integrates a starvation prevention mechanism based on Green Denial Counter (GDC) mechanism. This strategy tracks how many cycles each RSEN has been skipped and intervenes when necessary.

Recent work (e.g., [69]) highlights the importance of balancing efficiency and equity in signal control. While optimizing for high throughput is essential, it can unintentionally penalize low-saturation lanes. Our GDC-based intervention introduces a cycle-aware safeguard to ensure that no approach is indefinitely ignored, even if its traffic density is relatively low. The primary node maintains a GDC map (`GDC_map`) alongside the traffic density map (`traffic_density_map`). The decision-making steps of the algorithm are illustrated in Fig. 7. This flow diagram provides a high-level abstraction of how inputs such as traffic density and GDC status are processed to make adaptive signal decisions.

However, in real-world deployments, specific challenges arise, particularly when multiple traffic metrics such as vehicle count, traffic density, and waiting time yield conflicting indications. Our system addresses two such common conflict scenarios:

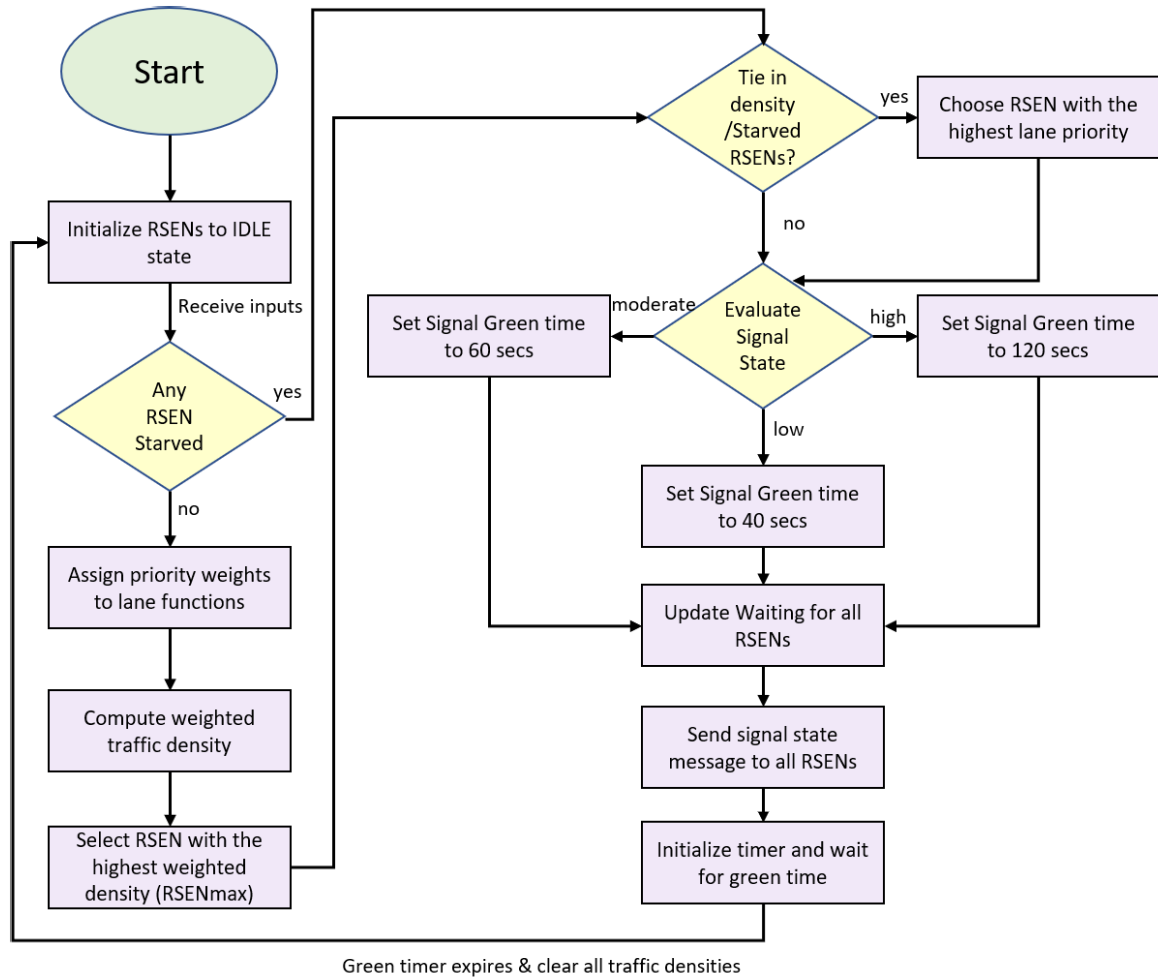


FIGURE 7. Workflow diagram of the proposed ATLC algorithm.

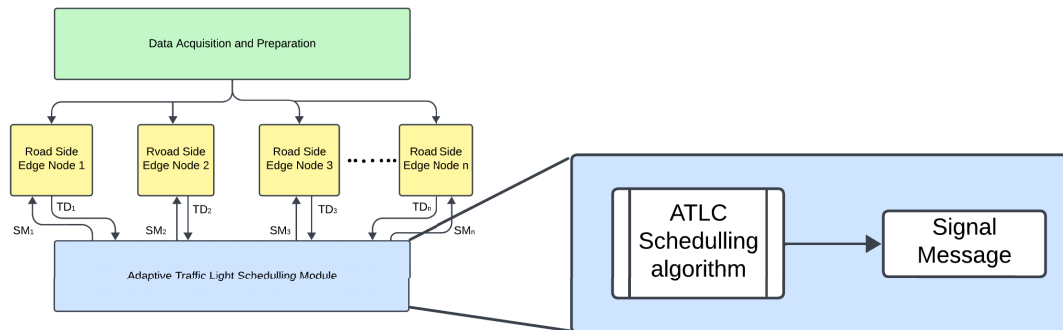


FIGURE 8. Adaptive Traffic Light Control Module.

• Case 1: Vehicle Count vs. Traffic Density.

A lane with many small vehicles (e.g., motorcycles, cars) may report high counts but low density, while a lane with fewer but larger vehicles (e.g., trucks) may have high density.

⇒ **Resolution (Density Based Scheduling):** The ATLC algorithm bases decisions on *traffic density*, not vehicle counts only. It incorporates PCE values

for each vehicle type, offering a more accurate measure of actual road occupancy (see equation 1). This resolves the first conflict by offering a more realistic indicator of congestion than vehicle count alone.

• Case 2: Traffic Density vs. Waiting Time.

A low-density lane may still experience long delays if it is consistently deprioritized.

⇒ Resolution (Starvation Prevention Mechanism):

To handle the second conflict, we incorporate a starvation prevention mechanism via the GDC. If a lane's GDC exceeds the predefined threshold `GDC_threshold`, it is immediately prioritized in the next cycle, regardless of its density. This mechanism ensures fairness and prevents extended delays for low-density lanes, helping maintain service equity across all approaches.

With these conflict resolution strategies in place, the ATLC algorithm executes each control cycle in a structured manner. Initially, all signal phases are set to idle, and two key maps are initialized: the `traffic_density_map`, which stores real-time density values received from each RSEN, and the `GDC_map`, which tracks how many consecutive cycles each RSEN has been denied a green signal.

At the beginning of each cycle, the algorithm first checks for starvation. If any RSEN has a GDC exceeding the predefined threshold `GDC_threshold`, that RSEN is immediately selected for service, overriding density-based selection to ensure fairness.

If no starvation condition is detected, the algorithm proceeds to compute a weighted traffic density for each RSEN, combining real-time congestion data and lane priority (left-turn, right-turn, or through). The RSEN with the highest weighted density is selected for the next green phase. In case of a tie, the lane with the higher predefined lane priority is chosen.

Once an approach (RSEN_max) is selected, the algorithm adaptively allocates green time: 120 seconds for high density, 60 seconds for moderate, and 40 seconds for low density. This dynamic adjustment ensures that heavily congested lanes receive sufficient clearance while minimizing delays for lighter traffic flows.

After serving the selected lane, its GDC value is reset to zero, while the GDC values of all other lanes are incremented by one to account for their continued denial, enabling the starvation prevention logic in future cycles.

Finally, the ATLC communicates the updated signal states to all RSENs through a compact 4-bit message containing RSEN_id, state, and a timer is initialized for the current green phase. When the green phase ends, the `traffic_density_map` is refreshed with new data from RSENs, and the cycle repeats.

This continuous, feedback-driven process allows the ATLC to adapt in real time to evolving traffic patterns, ensuring both efficiency and fairness. The complete operation of the ATLC algorithm can be summarized in the following steps:

Step 1. Initialization

- All traffic signals are set to the **idle state (00)**, meaning no lane currently has a green signal.
- A Green Denial Counter Map (`GDC_map`) is initialized with zeros for all RSENs to track how many times each lane is denied a green signal.
- The primary node collects the latest traffic density values from all RSENs and updates the `traffic_density_map`.

Step 2. Starvation Check

- The algorithm checks each RSEN's GDC value.
- If any RSEN's GDC exceeds the predefined threshold (`GDC_threshold`), that RSEN is marked as starved.
- The starved RSEN is immediately prioritized and set as `RSEN_max`, which will receive the next green signal.

Step 3. Lane Prioritization (if no RSEN is starved)

- For all RSENs, the weighted traffic density is calculated using equation 2:

$$\text{weighted_density} = \text{traffic_density} \times \text{priority_weight} \quad (2)$$

where the priority weights are:

- **Left-turn lane:** Priority weight = 3
- **Right-turn lane:** Priority weight = 2
- **Through lane:** Priority weight = 1

- The RSEN with the highest weighted density is selected as `RSEN_max`.
- In case of a tie, the lane with the highest lane function priority (e.g., left-turn > right-turn > through) is chosen.

Step 4. Adaptive Green Time Allocation

- Based on the weighted density of `RSEN_max`, the system allocates a green signal duration according to Table 9:
 - High density: state = 01, green time = 120 seconds.
 - Moderate density: state = 10, green time = 60 seconds.
 - Low density: state = 11, green time = 40 seconds.
- This ensures that highly congested lanes receive more green time, while lightly congested ones receive shorter cycles.

Step 5. GDC Update

- The GDC for `RSEN_max` is reset to 0, as it is being served.
- For all other RSENs (not selected), their GDC values are incremented by 1 to track consecutive denials.

Step 6. Signal State Communication

- The primary node sends a 4-bit **signal state message** {RSENid, State} to all RSENs, where:
 - i) 2 bits represent the RSEN ID (up to 4 lanes).
 - ii) 2 bits represent the signal state (idle, high, moderate, or low traffic mode), as given in Table 9.

Step 7. Timer Initialization

- A timer is set for the allocated green time (40/60/120 seconds).
- The system holds its state until the green phase ends.

Step 8. Clearing and Iteration

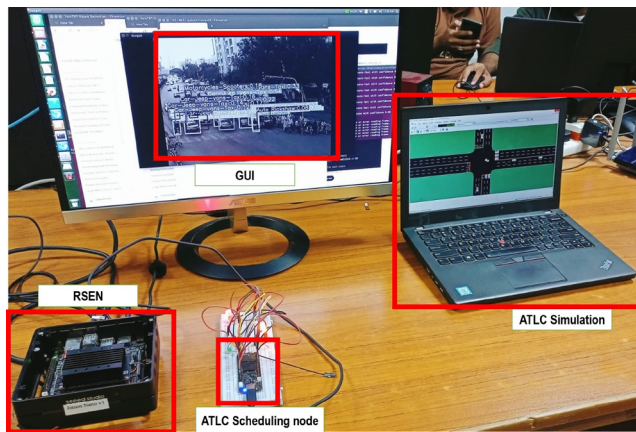
- After the green phase, the `traffic_density_map` is cleared.
- The system waits for updated traffic data from all RSENs.
- The cycle restarts from Step 2, ensuring continuous adaptation to real-time traffic conditions.

TABLE 9. Traffic density levels and Signal State encoding.

State		Code	Traffic Density Level	Traffic Density (t_d)	Description
Idle		00	-	-	traffic Signal is inactive
Active	State 1	01	Low	$t_d < 20$	traffic signal is active for 40 secs
	State 2	10	Moderate	$20 < t_d < 40$	traffic signal is active for 60 secs
	State 3	11	High	$t_d > 40$	traffic signal is active for 120 secs

D. HARDWARE IMPLEMENTATION OF RSEN AND ATLC

The lab setup of the proposed framework is depicted in Fig.9. This setup includes the RSEN node, ATLC scheduling node, GUI, along with the ATLC simulation for visualization.

**FIGURE 9.** Hardware setup of the proposed framework.

- **RSEN Node (Secondary-node):** The RSEN node, implemented on two different boards (Jetson Nano and Xavier), is responsible for real-time vehicle detection and traffic density estimation. The Intel 8265 wireless network adapter, placed below the heat sink of the Jetson device, facilitates efficient wireless communication between the RSEN nodes and the central primary node. This setup ensures high-speed data transmission and reliable performance in edge environments. The hardware specifications of Jetson Nano and Xavier NX are given in Table 10.
- **ATLC Scheduling Node (Primary-node):** The ATLC scheduling node is implemented on an H7 Portenta board due to its robust processing capabilities, low power consumption, and built-in connectivity features, making it an ideal option for edge deployment. This module receives traffic density updates from the RSEN nodes and determines the optimal traffic signal schedule. The H7 Portenta's built-in WiFi module enables seamless communication with the RSEN nodes, ensuring synchronized traffic flow. The hardware specifications of H7 portenta is given in Table 11
- **Graphical User Interface (GUI):** The GUI, displayed on the monitor, provides a visual representation of the detected vehicles and their classifications. This interface allows for real-time monitoring and verification of

the vehicle detection process, enhancing the system's usability and effectiveness.

- **ATLC Simulation:** The ATLC simulation, shown in the Fig.9, models the traffic light scheduling based on the aggregated information received from the RSEN nodes. This simulation validates the performance of the adaptive traffic light control algorithm in dynamically adjusting signal timings to minimize congestion.

The primary node (H7 portenta) acts as a central coordinator, receiving traffic density updates from all RSEN nodes (Jetson nano), and determining the optimal traffic signal schedule based on aggregated information, as depicted in Fig.10. This centralized decision-making ensures synchronized traffic flow and minimizes congestion by dynamically adjusting signal timing in response to real-time traffic conditions.

IV. RESULTS AND DISCUSSION

This section presents a multi-level evaluation of the proposed ATLC framework. Our aim is to trace performance from detection accuracy to edge feasibility to traffic flow outcomes, ensuring the proposed system is not only technically sound but also practically deployable in real-world urban settings. The evaluation proceeds across three tightly linked layers:

- **Vehicle Detection Validation:** We benchmark and evaluate the vehicle detection module on both public and real-world datasets to assess accuracy and inference speed, as detailed in the subsequent Section IV-A.
- **Edge Deployment Feasibility:** We identify the most deployment-ready YOLO variants suitable for resource-constrained edge hardware. Real-time performance is measured across edge platforms to confirm that the selected models maintain the required frame rates with high accuracy, as discussed in the subsequent Section IV-B.
- **Adaptive Traffic Control Effectiveness:** The validated vehicle detection pipeline is integrated with an adaptive control algorithm in a SUMO-based simulation, calibrated to real intersections. We quantify improvements in congestion reduction, delay minimization, and starvation prevention compared to traditional fixed-time baselines, as presented in the subsequent Section IV-C.

A. VEHICLE DETECTION VALIDATION

The vehicle detection pipeline was evaluated in both a baseline public data set and a custom field-collected traffic data set from multiple signalized intersections in Karachi, Pakistan.

Algorithm 1 Adaptive Traffic Light Control With Starvation Prevention and Tie-Breaker**Require:** A dictionary `traffic_density_map` where each entry is {RSEN id: traffic density}**Ensure:** Appropriate state for each traffic signal

```

1: Step 1: Initialization
2: Initialize all traffic signals to state 00 (idle)
3: Initialize GDC with zeros for all RSENs
4: while true do
5:   The scheduler receives traffic density from each RSEN
6:   Update traffic_density_map = {RSEN id: traffic density}
7:   Step 2: Starvation Check
8:   starved_RSEN = None
9:   for each RSEN id in GDC_map do
10:    if GDC[RSEN id] > GDC_threshold then
11:      starved_RSEN = RSEN id
12:      Break
13:    end if
14:  end for
15:  Step 3: Lane Prioritization (if no RSEN is starved)
16:  if starved_RSEN is not None then
17:    RSEN_max = starved_RSEN
18:  else
19:    Assign priority weights to lane functions:
20:    - Left-turn lane: Priority weight = 3
21:    - Right-turn lane: Priority weight = 2
22:    - Through lane: Priority weight = 1
23:    Calculate weighted traffic density for each RSEN:
24:    for each RSEN id in traffic_density_map do
25:      weighted_density = traffic_density_map[RSEN id] × priority_weight
26:      traffic_density_map[RSEN id] = weighted_density
27:    end for
28:    Find the RSEN_max with the highest weighted traffic density in traffic_density_map
29:    Tie-Breaker: If multiple RSENs have the same weighted_density, select the one with the highest lane function priority.
30:  end if
31:  Step 4: Adaptive Green Time Allocation
32:  if weighted traffic density is high then
33:    Set state of RSEN_max to 01 (green time = 120 secs)
34:  else if weighted traffic density is moderate then
35:    Set state of RSEN_max to 10 (green time = 60 secs)
36:  else
37:    Set state of RSEN_max to 11 (green time = 40 secs)
38:  end if
39:  Step 5: GDC Update
40:  for each RSEN id in GDC_map do
41:    if RSEN id == RSEN_max then
42:      Reset GDC[RSEN id] to 0
43:    else
44:      Increment GDC[RSEN id] by 1
45:    end if
46:  end for
47:  Step 6: Signal State Communication
48:  Send signal state message {RSEN id, state} to all corresponding secondary nodes
49:  Step 7: Timer Initialization
50:  Initialize timer to 0
51:  while timer < green time do
52:    Increment timer by 1 second
53:    Wait for 1 second
54:  end while
55:  Step 8: Clearing and Iteration
56:  Clear traffic_density_map and wait for new inputs
57: end while

```

All experiments were initially performed on a high-end GPU workstation (NVIDIA Tesla T4; 16 GB memory; 2,560 CUDA cores) to eliminate hardware bottlenecks while

comparing models, followed by edge-constrained validation on edge GPUs to ensure the selected models met both accuracy and efficiency targets of the edge environment.

TABLE 10. Hardware specifications of RSEN (secondary node).

Specifications	NVIDIA Jetson Nano	NVIDIA Jetson Xavier NX
Edge Accelerator	128 core NVIDIA Maxwell GPU	384 core NVIDIA Volta GPU with 48 Tensor cores
AI Performance	0.5 TFLOPs	1.3 TFLOPs
CPU	Quad Core ARM Cortex A57 MP Core processor	6 core NVIDIA Carmel ARM v8.2 64bit CPU 6MB L2+4MB L3
Memory	4GB 64-bit LPDDR4 25.6GB/s	8GB 128-bit LPDDR4 51.2GB/s
Edge Accelerator interface	PCIe	PCIe
Dimensions	69.6mm*45mm	69.6mm*45mm
Nominal Power envelope	5W-10W	10W-15W

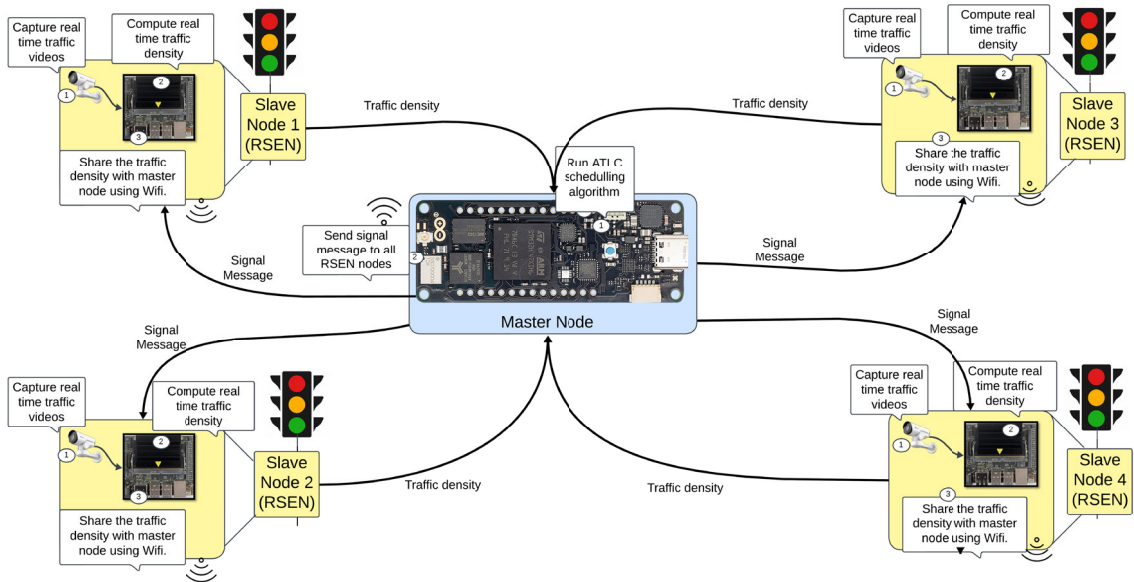


FIGURE 10. Data flow for the proposed ATLC framework.

TABLE 11. Hardware specifications of ATLC Module (Primary Node).

Specification	Details
Processor	ARM Cortex M7, ARM Cortex-M4 GPU
CPU clock	480MHz, 240MHz
Flash	16MB
SRAM	8MB, SDRAM
Power/Voltage	3.7-5V, Li-Pocell, 700mAh
Connectivity	WiFi, BLE, 10/100 EthernetPhy, USB, MIPIDSI, MPID-PHY
Sensors	Temperature, camera extension
Organization	Arduino

The validation was carried out in two stages:

- 1) We evaluated multiple YOLO-family variants across three color scales (RGB, gray, green) to identify model that retain accuracy when color information is degraded or illumination is low.
- 2) The selected variants from the previous stage were further evaluated on benchmark and real world traffic data, with and without augmentation, to assess robustness to spatial, temporal, and weather variability.

1) PERFORMANCE ANALYSIS OF YOLO VARIANTS ACROSS DIFFERENT COLOR SCALES

The vehicle detection module must handle varied visual conditions, bright daylight, deep shadows, and nighttime scenarios. To evaluate robustness, we tested YOLO variants across three color scales:

- RGB (Baseline): Typical daytime feed with full color detail.
- Grayscale: Simulates low-light or bandwidth-limited streams and explores whether removing color information would enhance accuracy and FPS, particularly in scenarios with reduced lighting.
- Greenscale: Simulating nighttime vision, we sought to evaluate the models' performance under low-light conditions, specifically when night vision cameras are used for traffic surveillance.

Experiments were conducted on real traffic video captured from the Allama Shabbir Usmani Road intersection. Four YOLO variantS were tested: YOLOv7-tiny [27],

YOLOv8-nano [66], YOLOv8-small [66], and YOLONAS-small [67]. Performance metrics include mean Average Precision (mAP) for detection accuracy (Equation 4) and Frames Per Second (FPS) for inference speed, Equation 3. Model file size (MB) is also reported as an indicator of memory footprint.

$$\text{Frames per second (fps)} = \frac{\text{Total number of processed frames}}{\text{Total processing time in seconds}} \quad (3)$$

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N AP_i \quad (4)$$

where: N is the number of classes, AP_i is the average precision for class i .

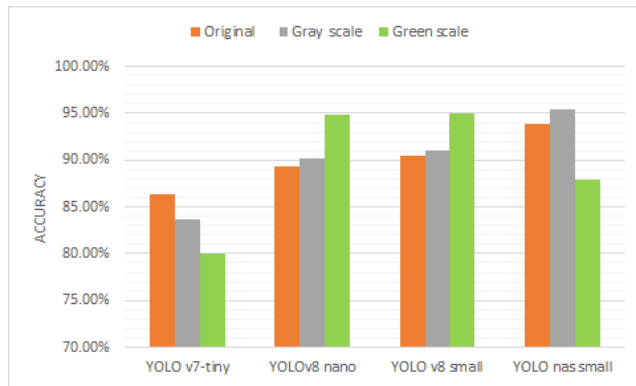


FIGURE 11. Comparison of accuracy among different YOLO Models on different color scales.

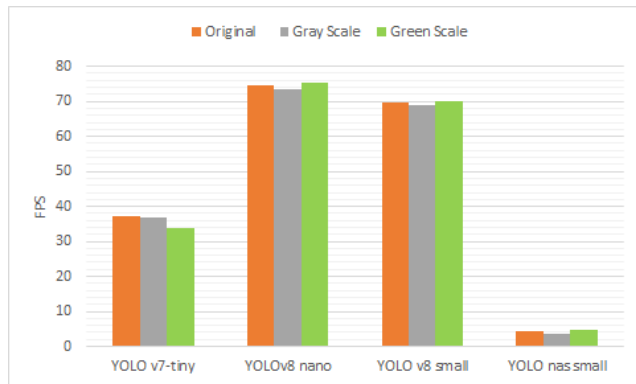


FIGURE 12. Comparison of FPS among different YOLO Models on different color scales.

Reducing model complexity typically trades off some detection accuracy [70], however, we mitigated this effect through a comprehensive data pipeline incorporating rigorous pre-processing, precise annotations, and targeted augmentations reflecting real-world variability (occlusions, lighting shifts, dense flows, and weather diversity), as discussed previously in Sec III-A. This enhanced the proposed system's ability to generalize and maintain high accuracy despite

its lightweight architecture. A comparison of the results in Table 12 yields the following observations:

- YOLONAS-small achieves the highest peak accuracy in Gray (95.4% mAP) and strong performance in RGB (93.9%), but is computationally heavy (244 MB) and slow (5 FPS), making it an impractical choice for edge inference.
- YOLOv8-nano (6 MB) delivers excellent speed (74.67 FPS RGB; similar across Gray/Green) while maintaining competitive accuracy (89–95% mAP across scales). Relative to YOLONAS-small, it is 41× smaller and 17× faster (RGB case) while losing only 4.6 percentage points in accuracy.
- YOLOv8-small provides a middle ground: higher accuracy than YOLOv8-nano in some color modes (e.g., Green 95.0% mAP) but at a 3.5× larger model size and reduced speed.
- YOLOv7-tiny remains a potential contender for resource-limited devices; although its accuracy lags, it offers consistent real-time FPS and a modest memory footprint.
- YOLOv8-nano and YOLOv8-small retain strong accuracy in the Green scale (94.9% and 95.0%, respectively), indicating resilience to spectral shifts typical of night-vision feeds.

It is important to note that model accuracy is not solely dependent on parameter count but also on the efficiency of its internal architecture. For instance, YOLOv8-nano, despite being lightweight (6 MB), leverages an optimized backbone with improved learning capacity and an enhanced detection head that collectively enable it to achieve good accuracy while maintaining real-time performance, as shown in Table 12. This demonstrates how architectural improvements can compensate for a smaller parameter size.

Furthermore, YOLOv8-nano and YOLOv8-small retain strong accuracy in the green scale (94.9% and 95.0%, respectively), highlighting their robustness to spectral variations common in low-light or night-vision traffic footage. The comparative accuracy and FPS trends across different YOLO variants and color modes are visualized in Fig. 11 and Fig. 12.

Based on the combined criteria of accuracy, speed, and memory footprint, YOLOv8-nano and YOLOv7-tiny emerge as the most suitable models for edge deployment in our vehicle detection module (see Sec. IV-B). Both offer favorable trade-offs, aligning well with the resource and energy constraints typical of edge computing environments.

2) AUGMENTATION AND GENERALIZABILITY STUDY

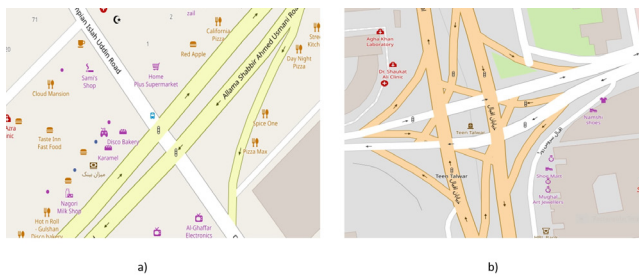
To assess the generalizability of our system, we conducted a comprehensive evaluation across multiple dimensions, focusing on its adaptability to varying traffic dynamics, weather conditions, and different times of the day. As part of this assessment, We evaluated the effect of augmentation on the two most optimal YOLO variants (identified in the previous Sec. IV-A1) under two regimes, unaugmented

TABLE 12. Performance of different YOLO vehicle detection models across varying color scales.

Metric	Color Scale	YOLOv7-tiny [27]	YOLOv8-nano [66]	YOLOv8-small [66]	YOLO-NAS-small [67]
Model Size (MB) ↓	—	11.8	6.0	21.5	244.4
Accuracy (mAP) ↑	RGB	86.40%	89.30%	90.50%	93.90%
	Gray	83.70%	90.20%	91.00%	95.40%
	Green	80.00%	94.90%	95.00%	87.90%
Speed (FPS) ↑	RGB	37.39	74.67	69.59	4.26
	Gray	36.83	73.56	69.10	3.70
	Green	33.95	75.36	70.07	4.81

and augmented, and assessed their ability to generalize across diverse scenarios. The key performance metric used was mean Average Precision (mAP), which quantifies the system’s detection accuracy under diverse conditions. Specifically, performance was measured on

- Two standard benchmarks (KITTI [53], PASCAL VOC [84]). These datasets contain diverse vehicle types, backgrounds, and environmental variations. This helped validate the model’s performance beyond the localized urban scenarios captured in our own dataset.
- Two structurally distinct real-world intersections in Karachi: Allama Shabbir Usmani and Teen Talwar. Allama Shabbir Usmani represents a conventional cross-type intersection with orthogonal lanes and service roads. The surrounding environment is a residential-commercial mix, with moderate pedestrian and vehicle density. In contrast, Teen Talwar is a roundabout-style junction in a high-density urban zone, marked by circular geometry. The area is surrounded by landmarks and commercial outlets with a significantly more chaotic and complex traffic scenario. The two intersections used in this study differ significantly in both road geometry and traffic flow behavior, as illustrated in the Fig. 13.

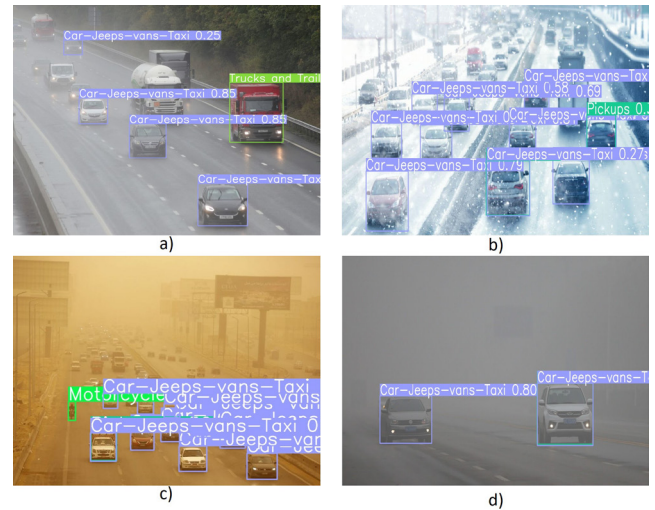

FIGURE 13. Comparison of the (a) Allama Shabbir Usmani intersection and the (b) Teen Talwar intersection in Karachi.

Importantly, Allama Shabbir Usmani was used as the primary training and evaluation site, whereas Teen Talwar served exclusively as an unseen testing location. This setup allowed us to rigorously assess the model’s generalization capability across urban settings with different traffic dynamics.

Performance results, reported as mean Average Precision (mAP) in Table13, demonstrate minimal degradation when moving from the training site to the unseen intersection.

Notably, the augmentation pipeline significantly narrows this gap. Across all cases, the augmented models significantly outperformed their unaugmented counterparts, demonstrating the effectiveness of our scenario-based augmentation strategy.

We also tested the performance under diverse weather conditions using the DAWN dataset [54], which is known for its extensive weather diversity, including fog, rain, dust, and low-visibility scenarios. The Fig. 14 illustrates that our context based augmentation strategy effectively prepared the model to handle weather related visual challenges, ensuring robust performance in real-world conditions.


FIGURE 14. a)rainy b)snowy c)sandy d)foggy: Sample vehicle detection results on DAWN dataset showing consistent model performance across different weather conditions.

In addition to weather testing, we also evaluated temporal robustness by sampling video across morning, afternoon, evening, and night, covering both peak and off peak demand, to assess temporal generalization (time of day variability). As shown in Figure15, the models consistently detect vehicles accurately across diverse lighting conditions and wide range of traffic volumes of TeenTalwar intersection, reflecting its strong adaptability in real-world deployments.

B. EDGE DEPLOYMENT FEASIBILITY

While high-end GPU performance provides baseline metrics, real-world traffic monitoring systems require deployment on resource-constrained edge devices for practical

TABLE 13. Cross-domain performance (mAP) of YOLOv7-tiny and YOLOv8-nano with and without augmentation.

Category	Dataset / Location	YOLOv7-tiny [27] (mAP) $\uparrow$		YOLOv8-nano [66] (mAP) $\uparrow$	
		Unaugmented	Augmented	Unaugmented	Augmented
Benchmark Datasets	KITTI [53]	67.67%	83.30%	72.20%	89.00%
	PASCAL VOC [89]	80.10%	86.50%	82.50%	95.00%
Real-World Intersections	Allama Shabbir Usmani	81.00%	86.40%	83.50%	89.30%
	Teen Talwar	62.00%	84.35%	75.00%	88.50%

TABLE 14. Cross-domain performance (mAP) of YOLOv7-tiny and YOLOv8-nano with and without augmentation.

Category	Dataset / Location	YOLOv7-tiny [27] (Unaugmented)	YOLOv7-tiny [27] (Augmented)	YOLOv8-nano [66] (Unaugmented)	YOLOv8-nano [66] (Augmented)
Benchmark Datasets	KITTI [53]	67.67%	83.30%	72.20%	89.00%
	PASCAL VOC [84]	80.10%	86.50%	82.50%	95.00%
Real-World Intersections	Allama Shabbir Usmani	81.00%	86.40%	83.50%	89.30%
	Teen Talwar	62.00%	84.35%	75.00%	88.50%

**FIGURE 15.** Sample vehicle detection results at the Teen Talwar intersection during (a) morning, (b) afternoon, (c) evening, and (d) night, demonstrating consistent model performance across varying times of day.

implementation. Based on the optimal YOLO variant identified in the Section IV-A1, YOLOv7tiny and YOLOv8nano, we evaluated these models' performance on embedded hardware platforms specifically designed for edge AI applications. Performance evaluation was conducted on two NVIDIA Jetson platforms: Jetson Nano and Jetson Xavier NX, details specified in Table 10. The lightweight models were prepared for edge deployment through TensorRT optimization to achieve real-time performance while maintaining detection accuracy [55]. Table 15 shows a slight decrease in accuracy compared to high-end GPUs, primarily due to the quantization and optimization processes involved in Jetson edge GPU deployments [55]. This is to satisfy real-time performance and energy efficiency requirements, which can marginally impact model accuracy. These results indicate that both the models maintains high detection quality and real-time response even under different road geometries and traffic flow patterns, supporting their applicability across

heterogeneous urban intersections with different traffic dynamics.

TABLE 15. Performance comparison of edge devices Jetson Nano and Jetson Xavier.

Intersection	Model	Accuracy (mAP) $\uparrow$	Jetson Nano (FPS) $\uparrow$	Jetson Xavier (FPS)
AllamaShabbirUsmani	YOLOv7-tiny [27]	82%	23	27
	YOLOv8-nano [66]	83.7%	—	35
Teen Talwar	YOLOv7-tiny [27]	78.4%	19	24
	YOLOv8-nano [66]	81.45%	—	33.5

1) WILCOXON SIGNED-RANK TEST FOR MODEL COMPARISON

To rigorously assess the performance differences between model A (YOLOv8-nano) and model B (YOLOv7-tiny), we conducted the **Wilcoxon signed-rank test** [71], a non-parametric statistical test suitable for comparing paired samples. This test evaluates whether there is a significant difference in the median **mean Average Precision (mAP)** scores of the two models across multiple runs, accounting for variability in performance.

The Wilcoxon signed-rank test is computed using the formula:

$$W = \sum \text{sgn}(x_i - y_i) \cdot R_i \quad (5)$$

where:

- x_i and y_i are the mAP scores of modelA and modelB for the i -th run,
- $\text{sgn}(x_i - y_i)$ is the sign function determining whether x_i is greater or smaller than y_i ,
- R_i is the rank of the absolute differences $|x_i - y_i|$.

A **low p-value** ($p < 0.05$) indicates a statistically significant difference between the models. The obtained p-value of **0.002** confirms a significant difference, favoring

YOLOv8-nano over YOLOv7-tiny. These results validate that the performance difference between the models is not due to random variation, reinforcing the superior accuracy of YOLOv8nano for real-time vehicle detection in resource-constrained environments.

C. EFFECTIVENESS OF ADAPTIVE TRAFFIC LIGHT CONTROL

To evaluate the effectiveness of the proposed framework on a simulated traffic network, we integrated the validated vehicle detection pipeline with the ATLC logic described in Algorithm 1. Our traffic simulation platform is Simulation of Urban Mobility (SUMO), which is an open-source microscopic traffic simulation package. The proposed ATLC scheduling algorithm was integrated with SUMO using the Traffic Control Interface (TraCI) API [72], allowing real-time interaction with simulation. TraCI facilitated data retrieval on vehicle densities and traffic flow, allowing the system to dynamically update signal timings. Meanwhile, we used Pytorch 1.8.1, one of the most popular deep learning frameworks, for our experiments.

The simulation parameters, including vehicle types, traffic flow, and the road network, are configured using real-world traffic data collected by the vehicle detection module of the proposed system. This ensures that the simulations closely reflect actual traffic conditions. The road network of Allama Shabbir Usmani Road, Karachi, was first exported using the OSM (Open Street Map) Web Wizard to obtain structured road data with accurate lane configurations and intersections, as shown in Fig. 16a. To ensure accuracy, the OSM map was cross-verified with Google Maps as shown in Fig. 16b, allowing for validation of road alignments and intersections against real-world geographical data. Next, the network was refined and edited in NetEdit, SUMO's built-in network editor, where lane adjustments, signal placements, and additional refinements were incorporated to enhance realism, as depicted in Fig. 16c. Finally, the processed road network was imported into the SUMO simulation environment for traffic flow analysis (Fig. 16d). The traffic flows were generated using real-world vehicle counts extracted from traffic videos collected from Allama Shabbir Usmani Road. These counts coupled with PCE values ensure a realistic representation of congestion. This systematic approach ensures a highly accurate traffic simulation for analyzing congestion and optimizing adaptive traffic signal control strategies.

For our experiments, traffic flow data was selected from two time intervals on a typical weekday in July 2024: 10:00–12:00, representing medium flow (off-peak hours), and 18:00–20:00, representing high flow (peak hours). These intervals correspond to flat and peak traffic scenarios in the simulation environment. The traffic demand distribution during these periods is illustrated in Fig. 17. The proposed ATLC algorithm is evaluated using key metrics within the SUMO environment, including average traffic density (ATD)

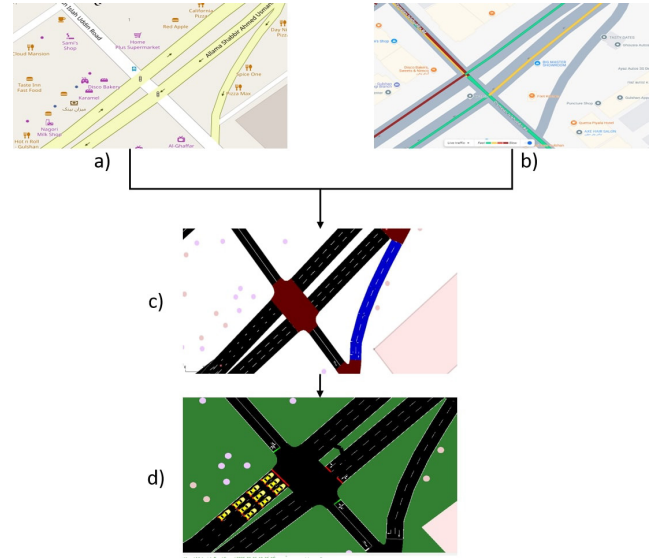


FIGURE 16. Road Network of Allama Shabbir Usmani Road a) OSM view b) Satellite view c) netedit view d) SUMO View.

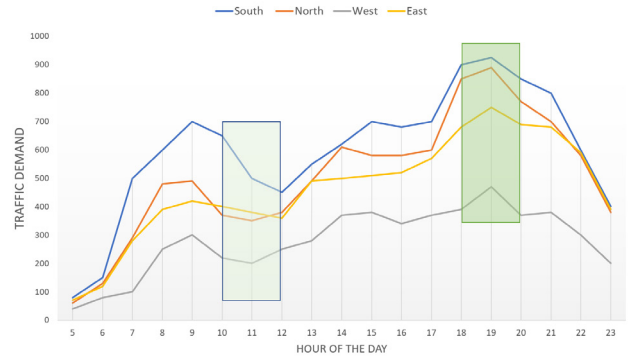


FIGURE 17. The traffic demand on a typical workday. The light green and dark green shaded regions highlight periods of medium (13:00–15:00) and high (18:00–20:00) traffic density, respectively.

(see Equation 6) [40] and average waiting time (AWT)(see Equation 7).

The traffic density for each lane is calculated directly from the equation:

$$TD = \left(\frac{\sum \text{vehicle length} + \sum \text{gaps between vehicles}}{\text{total length of the observed road section}} \right) \times 100 \quad (6)$$

where:

- $\sum \text{vehicle length}$ is the total length occupied by all vehicles on the observed road section.
- $\sum \text{gaps between vehicles}$ is the inter-vehicle distance.
- Total length of the observed road section is the length of the road section being analyzed.

$$AWT = \frac{\sum \text{traci.vehicle.getWaitingTime(laneID)}}{\text{Number of Vehicles per Approach}} \quad (7)$$

The results presented in Fig. 18 are obtained from traffic simulations in SUMO. The comparison of traffic density at

south, north, west and east approaches under traditional and the proposed ATLC management strategies reveals a clear advantage for our adaptive approach. For all approaches, the adaptive strategy shows a significant reduction in traffic density. Specifically, the south approach sees a reduction of 16.25% in traffic density, the north approach observes 23.01% reduction, 33.38% and 25.42% reduction in traffic densities of the west and east approaches respectively. These results indicate that the adaptive traffic signal scheduling algorithm effectively reduces congestion by adjusting green times based on real-time traffic densities.

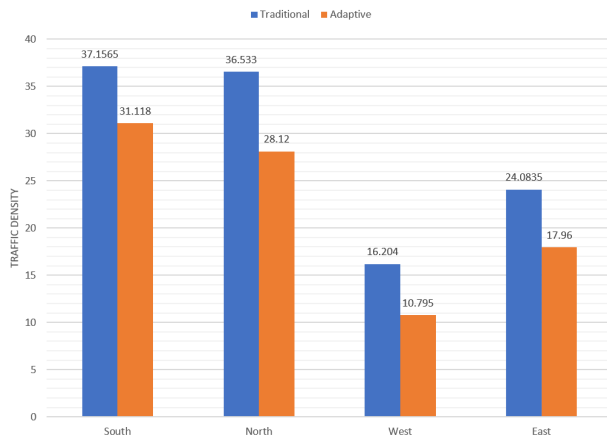


FIGURE 18. Comparison of average traffic density under traditional and the proposed ATLC.

Following the reduction in traffic density observed under the adaptive signal control system, we also analyzed its impact on average waiting time across all four approaches. As shown in Figure 19, the proposed ATLC algorithm significantly reduced waiting times across all approaches, with percentage reductions ranging from 16.6% to 23.4%.

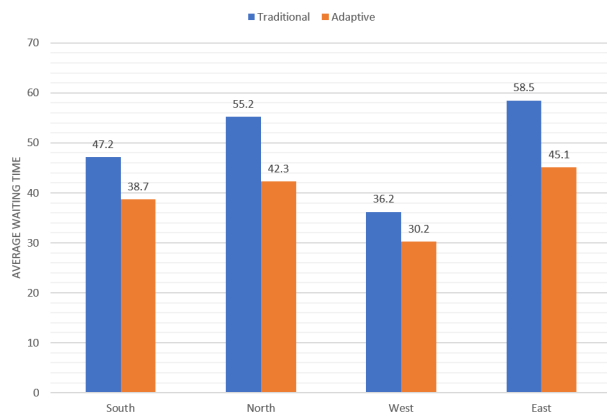


FIGURE 19. Comparison of average waiting time under traditional and the proposed ATLC.

Another key challenge in the proposed ATLC system is the prevention of traffic starvation, particularly for low-density approaches. To address this, the proposed starvation

prevention mechanism based on GDC logic, was integrated into the ATLC framework within the SUMO simulation environment.

The logic continuously operates in a loop to monitor each phase (RSEN). For every phase, the GDC is incremented whenever the phase is skipped (i.e., not granted green) despite having waiting vehicles. This is determined by checking whether the number of waiting vehicles on the phase is greater than zero and yet the current signal phase does not correspond to that direction. Once the phase is eventually served, the counter is reset. If the GDC exceeds the predefined threshold (GDC_threshold), the phase is flagged as starved and immediately assigned green in the next signal cycle, even if other approaches have higher densities.

This starvation logic was implemented using the SUMO's Traffic Control Interface (TraCI). The following metrics and their associated TraCI commands were used to monitor and manage traffic phases during simulation:

- **Current active signal phase:** Used to check which phase is currently green
`traci.trafficlight.getPhase(tlID)`
- **Number of waiting vehicles per lane:** Used to detect waiting vehicles on each approach
`traci.lane.getLastStepVehicleNumber(laneID)`
- **Current waiting time per vehicle:** Used to check how long a vehicle has been stopped
`traci.vehicle.getWaitingTime(vehicleID)`
- **Manual phase assignment (when starved):** Used to assign green to a starved phase
`traci.trafficlight.setPhase(tlID, phaseIndex)`

It is important to note that starvation detection is highly sensitive to the chosen value of GDC_threshold. Instead of selecting this threshold arbitrarily, we conducted a series of controlled simulations using a generic four-way signalized intersection in SUMO to empirically determine the most effective value. For each threshold setting, we evaluated three key performance indicators:

- Average waiting time across all approaches
- Maximum waiting time per approach (worst-case delay)
- Number of forced green assignments due to starvation intervention.

The Table 16 summarizes the observed results from our simulations:

TABLE 16. Impact of GDC threshold on waiting times and number of forced starvation resolutions.

GDC threshold	Max Waiting Time(s) per Phase	Avg Waiting Time (s)	Forced Starvation Resolutions
2	N: 38, E: 35, S: 33, W: 36	35.5	8
3	N: 42, E: 39, S: 40, W: 43	41	4
4	N: 46, E: 44, S: 45, W: 47	45.5	2
5	N: 50, E: 48, S: 49, W: 50	49.3	1

The results presented in Table 16, demonstrate the trade-off involved in selecting an appropriate GDC threshold. As the `GDC_threshold` increases, average waiting time also increases due to delayed interventions. Lower thresholds result in more frequent interventions, while higher thresholds reduce responsiveness to low-demand phases. Based on the results in the observation table, a GDC threshold of 3 was selected, as it provided a balanced trade-off, being neither too low (causing excessive interventions) nor too high (resulting in prolonged delays).

Having selected an appropriate value for `GDC_threshold` (3), we evaluated the effectiveness of the starvation prevention mechanism. Table 17 presents phase-wise observations, indicating that starvation prevention was activated selectively, only when necessary to avoid excessive denial of green time to lower-density approaches. This confirms the effectiveness of the proposed starvation prevention mechanism.

TABLE 17. Phase-wise starvation detection outcomes at GDC threshold = 3, showing intervention triggered only for low-density signal phases.

Signal Phase	Traffic Density	Average Waiting time	Max consecutive Green Denials	Starvation Intervention Triggered
South	31.118	38.7	2	No
North	28.12	42.3	2	No
West	10.795	30.2	3	Yes(1 time)
East	17.96	45.1	3	Yes(2 times)

V. COMPARATIVE ANALYSIS OF ITS SOLUTIONS

This section provides a detailed analysis of our work in comparison with recent studies on two levels. First, we compare our proposed technique with recent computer vision-based methods for traffic parameter computation, as presented in Table 18. Second, we conduct an in-depth comparison of our proposed framework with state-of-the-art methodologies used for adaptive traffic scheduling, as presented in Table 19. This dual-level comparison enhances the clarity and significance of our contributions.

A. COMPARISON OF RECENT CV-BASED TECHNIQUES FOR TRAFFIC PARAMETER COMPUTATION

Computer vision (CV)-based methods have been extensively utilized for traffic parameter estimation, including vehicle detection, traffic density estimation, road occupancy measurement, and adaptive traffic signal control. This Table 18 compares various existing studies with the proposed work, in terms of edge deployment, traffic parameter computation, methodologies, datasets, platforms, computational complexity and accuracy.

- **Edge Deployment:** As indicated in third column (Edge Deployment) of Table 18, a key strength of our work is the successful deployment of an edge-based solution, which is critical for real-time applications in ITS. Many other works, such as [9], [11], [29], [31], [32], [35], [74] and [75] rely on cloud-based or high-performance GPUs like NVIDIA Tesla, which, while powerful, may

not be practical for deployment in resource-constrained environments like smart cities.

- **Comprehensive Feature Set:** As shown in the fourth, fifth, sixth and seventh column of Table 18 our work addresses a broader spectrum of ITS requirements by incorporating road occupancy and traffic light scheduling, unlike many other works that focus solely on vehicle detection ([9], [10], [11], [29], [30], [34], [37], [38], [39] and [76]). This comprehensive approach improves the practical utility of our system in managing urban traffic more effectively.
- **Computational Complexity:** As indicated in column eleventh and twelfth of the Table 18, high computational power results in high power consumption, making large models despite of high performance, unsuitable for deployment in resource-constrained environments. Large models like YOLOv4 (>250 MB) in [30] and ensemble CNNs (244 MB) in [36] require significant processing power, limiting their feasibility for real-time edge applications. In contrast, the proposed framework minimizes computational complexity by using lightweight YOLO models (YOLOv7-tiny: 11.8 MB, YOLOv8-nano: 6 MB) optimized for edge devices like Jetson Nano and Xavier.
- **Accuracy and Speed:** As highlighted in thirteenth and fourteenth column of the Table 18, our work demonstrates a balanced trade-off between speed and accuracy, achieving 82% mAP with YOLOv7-Tiny at 27 FPS on Xavier and 23 FPS on Nano, and an even higher accuracy of 83.70% mAP with YOLOv8-Nano at 35 FPS on Xavier NX. In comparison, the study by [36] achieves a higher accuracy of 96.10% with YOLOv5 but at the cost of using a much larger model and a more powerful GPU, which limits its applicability in real-time edge scenarios.
- **Handling Undisciplined Traffic:** It is evident from Table 18 that very less work has been reported in ITS for handling undisciplined traffic. For example, [30], [35], [36] and [40] use standardized datasets like MS COCO, KITTI and FLIR, which, while valuable, but can only handle discipline traffic. Our use of real-time traffic videos for training and evaluation is particularly noteworthy, ensuring that our system is tuned for real-world conditions. To handle undisciplined traffic, we first developed a comprehensive dataset by capturing real traffic videos from Karachi, Pakistan. This dataset includes annotated images that reflect key characteristics of undisciplined traffic, such as lane-less movement, diverse vehicle types, frequent occlusions, and high-density conditions. Additionally, we applied various data augmentation techniques, such as mosaic augmentation and cutout augmentation, to improve generalization across complex traffic conditions. Additionally, we applied various data augmentation techniques, such as mosaic augmentation and cutout augmentation, to improve generalization across complex

TABLE 18. Comparative analysis of CV-based techniques for traffic parameter computation.

Traffic	Ref. Year	Edge Dep.	Veh. Det.	Road Occ.	Traffic Light Sched.	Other	Dataset	Platform	Algorithm	Size (MB)↓	Params (M)↓	Speed (FPS)↑	Accuracy (mAP)↑
Disciplined	[40], 2025	✓	✓	×	×	License plate recog.	robotflow DATASETS	Raspberry Pi	YOLOv8, YOLO11	—	—	5	95%
	[33], 2023	×	✓	×	×	—	KITTI	Mmdet (GPU)*	Resnet50	—	—	—	82.12%
	[77], 2023	×	✓	×	×	—	COCO, PKU, DAWN	T4 GPU	YOLOv5-s	14.1	7.2	—	COCO (52.31%), PKU (99.2%), DAWN (29.66%)
	[38], 2024	×	✓	×	×	—	COCO	NVIDIA GTX	DETR-SPP	—	35.7	53	51.31%
	[76], 2024	×	✓	×	×	—	Dawn COCO	—	YOLOv5	—	—	35.8	92.18%
	[74], 2024	×	✓	×	×	Vehicle counting	COCO, BDD100K	GPU	YOLOv5l	90.1	46.5	—	58.9%
	[75], 2025	×	✓	×	×	—	KITTI, Cityscapes	NVIDIA TITAN	Modified YOLOv7	—	66	0.04s/img	80.54%
	[29], 2022	×	✓	×	×	—	Cityscapes	Cloud (Train), NVIDIA PASCAL GPU (Inf.)	Improved YOLOv4	>250	—	31	86%
	[30], 2022	✓	✓	×	×	—	KITTI	NVIDIA Jetson	Channel pruned YOLOv4	24.48	8M	40.6 (Xavier), 9.7 (Nano)	82%
	[31], 2023	×	✓	×	×	—	Agroverse	8 cores processor with NVIDIA GP100L	YOLOv7	—	36.9	—	57.89%
	[32], 2022	×	✓	×	×	—	MS COCO, Agroverse	NVIDIA Tesla GPU	Improved YOLOv5	—	0.16	—	58%, 35%
	[47], 2023	×	✓	×	✓	—	—	Simulation (NVIDIA Jetson Nano)	YOLOv8	—	25.9M	—	—
Undisciplined	[35], 2022	×	✓	✓	×	—	FLIR thermal, FLIR RGB, MB7500, KITTI	Google Colab	Ensemble CNN	244	—	0.054	94% (FLIR Thermal), 93% (FLIR RGB), 89% (MB7500), 91% (KITTI)
	[36], 2023	×	✓	×	×	—	KITTI	NVIDIA GeForce RTX	Improved YOLOv5	>41MB	—	202	96.1%
	[18], 2023	×	✓	×	×	—	DTLD	Quadro P600 GPU	Swin Transformer	—	48M	17	91%
	[34], 2021	×	✓	×	×	—	DHAKA AI	NVIDIA GeForce RTX	YOLOv3	246.5	—	57	45.66%
	[37], 2023	✓	✓	×	×	—	DTLD	NVIDIA Tx2	Ensemble CNN	—	50.5614M	15	66.85%
	[39], 2021	✓	✓	×	×	—	DHAKA AI, IITM HETRA	GTX 1070 (Train), RPi 4 (Inf.)	DhakaNet	—	—	0.15 (DHAKA AI), 0.20 (IITM HETRA)	9.9% (DHAKA AI), 95% (IITM HETRA)
	Our work [Edge version]	✓	✓	✓	✓	—	Real Traffic Videos	NVIDIA Xavier NX, NVIDIA Jetson Nano	YOLOv7-tiny, YOLOv8	11.8, 6	6, 3	27 (Xavier NX, YOLOv7), 23 (Nano, YOLOv7), 35 (Xavier NX, YOLOv8)	83.70%

traffic conditions. The Mosaic Augmentation helped to expose the model to diverse vehicle arrangements. The Cutout Augmentation helped to simulate occlusions, improving the model's ability to detect partially visible vehicles. Furthermore, we implemented YOLOv7-tiny [27], YOLOv8 [66], and YOLO-NAS [67] for real-time vehicle detection as YOLOv7 Tiny and YOLOv8 utilize Feature Pyramid Networks (FPN) and Path Aggregation Networks (PAN), enabling effective detection of both small and large vehicles in highly dense and occluded traffic scenes.

This work demonstrates the successful real-time edge deployment of compact models on NVIDIA Xavier NX and Jetson Nano, making it highly suitable for resource-constrained environments. The use of lightweight models such as YOLOv7-Tiny and YOLOv8-Nano offers an excellent balance of speed and accuracy, achieving competitive performance with up to 83.70% mAP while maintaining high FPS. Furthermore, this work presents a comprehensive ITS solution that addresses multiple aspects such as vehicle detection, road occupancy, and traffic light scheduling, making it highly applicable to smart city initiatives. The robustness of the model is further validated by training

in real-world traffic videos, ensuring its effectiveness in practical scenarios [46].

B. COMPARISON ON ADAPTIVE TRAFFIC LIGHT CONTROL TECHNIQUES

Table 19 presents a comparative analysis of recent methodologies for adaptive traffic light control. While several studies have introduced novel approaches, the majority remain confined to simulation-based evaluations, limiting their real-world applicability.

Learning-based methods, particularly Reinforcement Learning (RL), have been extensively explored for traffic signal optimization. However, RL-based solutions face significant challenges in real-world deployment due to their computational demands, reliance on extensive training data, and difficulty in adapting to unpredictable traffic variations [79]. The limitations of RL-based methods become more evident when transitioning from simulation to real-world deployment since real-world traffic is influenced by many dynamic factors that current RL-based models often overlook. For example, extreme weather such as heavy rain reduces visibility and vehicles move more cautiously, conditions under which RL agents, typically trained in ideal

TABLE 19. Comparison of different traffic signal management approaches.

Reference, Year	Contribution	Limitations	Simulation Platform	Algorithm
[43], 2025	A graph-based learning approach for Traffic Signal Management	Tested in idealized simulation environment and not in complex traffic scenario	CityFlow	Learning Based
[78], 2024	Provide a rich, diverse dataset for training and evaluating traffic signal control algorithms, particularly for reinforcement learning	Does not provide a detailed evaluation of end-to-end traffic signal control systems in real-world settings, focusing instead on synthetic data.	CARLA simulator	Learning Based
[47], 2023	Emergency vehicle communication with adaptive traffic lights	Simulation-only, no edge deployment for real-world validation	-	Rule Based
[44], 2023	An improved deep reinforcement learning approach for traffic signal control that balances state and reward consistency	Simulation-only, no edge deployment for real-world validation	SUMO	Learning Based
[48], 2022	Using a fuzzy inference system, the system dynamically adjusts traffic signal based on arrival flow rates	Simulation-only, no edge deployment for real-world validation	SUMO	Rule Based
[41], 2019	Optimize signal efficiency by adjusting traffic light phase durations based on past traffic patterns.	Simulation-only, no edge deployment for real-world validation	SUMO	Learning Based
Our Work	Adaptive traffic management system designed for undisciplined traffic scenarios with edge deployment	High energy consumption of RSEN	NVIDIA Jetson Xavier	Rule Based

settings, struggle to perform effectively. Similarly, traffic volume also varies across the day and week, shifting from light to highly congested. Moreover, most RL models are trained using standard vehicle types, ignoring the wide variety of vehicles seen in local traffic, like rickshaws, motorcycles, minibuses, or overloaded trucks, which can vary significantly from region to region. These limitations make it hard for RL techniques to generalize well to real-world, region-specific traffic conditions [80], [81]. Further, the high computational requirements severely impact real-time decision-making of RL. In multi-phase intersections, the state and action spaces grow exponentially as the number of lanes, directions, and vehicle movements increase. For instance, at a standard four-way intersection, the RL agent must evaluate a vast range of states—defined by vehicle counts, lane occupancies, and waiting times—to select the optimal signal phase. This results in high inference latency and poses a challenge to real-time responsiveness [82]. Another major challenge lies in designing reward functions that balance competing traffic objectives, such as minimizing delay, ensuring safety, and maintaining fairness [83]. These constraints explain why most RL-based systems remain confined to ideal simulations and lack viability for scalable, real-world traffic signal management.

The Table 19 reveals that most of the work reported in adaptive traffic light control is confined to simulation environments, making it unsuitable for practical deployment in real-world traffic scenarios. In contrast, our contribution

addresses this gap by proposing a system that is not only validated through simulations but also designed on edge hardware making it viable for real world deployment. Moreover, our work differentiates itself by integrating a CV-based adaptive traffic management system designed specifically for undisciplined traffic scenarios. By leveraging lightweight YOLO models and a computationally efficient rule-based approach for the ATLC module, our system achieves both high speed and accuracy, enabling rapid adaptation to dynamic traffic conditions. Furthermore, our system minimizes latency by eliminating reliance on cloud-based processing, making it more suitable for real-world urban traffic management. However, the energy consumption of the Road Side Edge Node (RSEN) remains a limitation, which will be addressed in future optimizations.

VI. LIMITATIONS AND FUTURE WORK

While the proposed ATLC framework demonstrates robust performance under diverse conditions, several avenues remain open for further exploration and enhancements:

- 1) **Power-Aware Resource-Sensitive Edge Networks (RSEN):** Even though the current proposed framework utilizes edge GPUs optimized for real-time inference, large-scale smart city deployments will require significant power efficiency improvements. A promising future direction is to explore advanced power optimization techniques for RSEN, including approximate computing approaches [85], Dynamic Voltage and Frequency

Scaling (DVFS) and mixed-precision inference. These strategies can substantially reduce energy consumption while maintaining real-time responsiveness, enabling deployment on energy-constrained platforms such as microcontroller units (MCUs), NPUs, or FPGA-based nodes.

- 2) **Online and Continual Learning:** The current detection models are statically trained and deployed on edge nodes. Future work will focus on integrating online learning and continuous adaptation mechanisms (e.g., SMTC-CL: Selective Multi-Task Coordination for Continuous Learning [88]) to handle evolving traffic dynamics, infrastructure changes, or newly introduced vehicle types. This will ensure long-term adaptability by enabling the system to autonomously adapt to changing environments while preserving prior knowledge.
- 3) **Meta-Learning for Label Noise:** While our dataset was annotated manually using a rigorous multi-stage protocol with cross-checking and quality control, future large-scale datasets may necessitate partial automation of annotations, which can introduce label noise. A promising future direction is to explore Meta-Learning Guided Label Noise Distillation (MLG-LND) techniques [87], where a meta-teacher network can filter or reweight uncertain samples, enhancing detection stability and reducing performance degradation caused by noisy labels.
- 4) **Multimodal fusion** The current framework is predominantly vision-centric, relying solely on camera-based inputs for decision-making. In the event of camera malfunctions or degraded visual conditions (e.g., heavy rain, fog, or glare), this dependency may compromise perception reliability. A promising future direction is to enhance robustness through multimodal fusion, combining visual detections with low-bandwidth wireless and IoT traffic telemetry (e.g., probe vehicle data, V2X beacons, radar signals). Leveraging Virtual Signal Large Model (VSLM)-inspired few-shot wideband signal modeling [86] could enable reliable traffic state estimation even when visual channels are impaired.

VII. CONCLUSION

This research presents a comprehensive and adaptive traffic management framework designed to mitigate urban traffic congestion using real-time computer vision and deep learning. The proposed vehicle detection module demonstrates high accuracy and resilience in complex traffic scenarios, including occlusions, dense flows, and undisciplined driving behavior. Leveraging lightweight yet powerful object detection models—specifically YOLOv7-tiny and YOLOv8-nano—the system achieves a mean Average Precision (mAP) of up to 90 and real-time performance of 74 FPS. The Adaptive Traffic Light Control (ATLC) algorithm dynamically adjusts signal timings based on real-time traffic density and lane priorities. It incorporates starvation prevention and a tie-breaking mechanism to ensure fairness, reducing

intersection-level congestion by up to 33%, as validated through simulations and real-world testing. To ensure reliability under real-world conditions, a context-driven augmentation pipeline was implemented, simulating diverse weather effects (rain, fog, snow, dust) and lighting variations (day, dusk, night), enabling strong generalization across locations and conditions. The framework was successfully deployed on edge-friendly hardware platforms including Jetson Nano, Xavier NX, and Portenta H7. Its real-time performance across these devices confirms the system's feasibility and effectiveness for practical, scalable urban traffic management.

ACKNOWLEDGMENT

The research findings presented in this article are solely the author(s)' responsibility.

REFERENCES

- [1] F.-Y. Wang, "Parallel control and management for intelligent transportation systems: Concepts, architectures, and applications," *IEEE Trans. Intell. Transp. Syst.*, vol. 11, no. 3, pp. 630–638, Sep. 2010, doi: [10.1109/TITS.2010.2060218](#).
- [2] J. Zhang, F.-Y. Wang, K. Wang, W.-H. Lin, X. Xu, and C. Chen, "Data-driven intelligent transportation systems: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 12, no. 4, pp. 1624–1639, Dec. 2011, doi: [10.1109/TITS.2011.2158001](#).
- [3] T. Yuan, W. Da Rocha Neto, C. E. Rothenberg, K. Obraczka, C. Barakat, and T. Turetli, "Machine learning for next-generation intelligent transportation systems: A survey," *Trans. Emerg. Telecommun. Technol.*, vol. 33, no. 4, pp. 1–18, Dec. 2021, doi: [10.1002/ett.4427](#).
- [4] A. N. Oluwatobi, A. O. Tayo, A. T. Oladele, and G. R. Adesina, "The design of a vehicle detector and counter system using inductive loop technology," *Proc. Comput. Sci.*, vol. 183, pp. 493–503, Jan. 2021, doi: [10.1016/j.procs.2021.02.089](#).
- [5] L. A. Klein, "Roadside sensors for traffic management," *IEEE Intell. Transp. Syst. Mag.*, vol. 16, no. 4, pp. 21–44, Jul. 2024, doi: [10.1109/MITS.2023.3346842](#).
- [6] L. A. Klein, *ITS Sensors and Architectures for Traffic Management and Connected Vehicles*. Boca Raton, FL, USA: CRC Press, 2017.
- [7] A. Hauli, R. Kavalier, and P. Varaiya, "Wireless magnetic sensors for traffic surveillance," *Transp. Res. C, Emerg. Technol.*, vol. 16, no. 3, pp. 294–306, Jun. 2008, doi: [10.1016/j.trc.2007.10.004](#).
- [8] E. Dilek and M. Dener, "Computer vision applications in intelligent transportation systems: A survey," *Sensors*, vol. 23, no. 6, p. 2938, Mar. 2023, doi: [10.3390/s23062938](#).
- [9] Q. Fan, Y. Li, M. Deveci, K. Zhong, and S. Kadry, "LUD-YOLO: A novel lightweight object detection network for unmanned aerial vehicle," *Inf. Sci.*, vol. 686, Jan. 2025, Art. no. 121366, doi: [10.1016/j.ins.2024.121366](#).
- [10] M. Bakirci, "Enhancing vehicle detection in intelligent transportation systems via autonomous UAV platform and YOLOv8 integration," *Appl. Soft Comput.*, vol. 164, Oct. 2024, Art. no. 112015, doi: [10.1016/j.asoc.2024.112015](#).
- [11] C. Liu, X. Liu, S. Chen, X. Zuo, B. Qiao, and W. Zhang, "A high-precision vehicle detection model based on focused feature extraction and multi-scale feature fusion for remote sensing images," *Comput. Electr. Eng.*, vol. 123, Apr. 2025, Art. no. 110065, doi: [10.1016/j.compeleceng.2025.110065](#).
- [12] X. Zhou, R. Ke, H. Yang, and C. Liu, "When intelligent transportation systems sensing meets edge computing: Vision and challenges," *Appl. Sci.*, vol. 11, no. 20, p. 9680, Oct. 2021, doi: [10.3390/app11209680](#).
- [13] A. Ahmed, F. Outay, M. U. Farooq, S. Saeed, M. Adnan, M. A. Ismail, and A. Qadir, "Real-time road occupancy and traffic measurements using unmanned aerial vehicle and fundamental traffic flow diagrams," *Pers. Ubiquitous Comput.*, vol. 27, no. 5, pp. 1669–1680, Aug. 2023, doi: [10.1007/s00779-023-01737-w](#).
- [14] D. R. Aleko and S. Djahel, "An efficient adaptive traffic light control system for urban road traffic congestion reduction in smart cities," *Information*, vol. 11, no. 2, p. 119, Feb. 2020, doi: [10.3390/info11020119](#).

- [15] H. Sajid, S.-C.-S. Cheung, and N. Jacobs, "Motion and appearance based background subtraction for freely moving cameras," *Signal Process., Image Commun.*, vol. 75, pp. 11–21, Jul. 2019, doi: [10.1016/j.image.2019.03.003](https://doi.org/10.1016/j.image.2019.03.003).
- [16] P. E. Rybski, D. Huber, D. D. Morris, and R. Hoffman, "Visual classification of coarse vehicle orientation using histogram of oriented gradients features," in *Proc. IEEE Intell. Vehicles Symp.*, San Diego, CA, USA, Jun. 2010, pp. 921–928, doi: [10.1109/IVS.2010.5547996](https://doi.org/10.1109/IVS.2010.5547996).
- [17] R. Lienhart and J. Maydt, "An extended set of Haar-like features for rapid object detection," in *Proc. Int. Conf. Image Process.*, vol. 1, Rochester, NY, USA, Sep. 2002, pp. 1900–1903, doi: [10.1109/ICIP.2002.1038171](https://doi.org/10.1109/ICIP.2002.1038171).
- [18] P. Deshmukh, G. S. R. Satyanarayana, S. Majhi, U. K. Sahoo, and S. K. Das, "Swin transformer based vehicle detection in undisciplined traffic environment," *Expert Syst. Appl.*, vol. 213, Mar. 2023, Art. no. 118992, doi: [10.1016/j.eswa.2022.118992](https://doi.org/10.1016/j.eswa.2022.118992).
- [19] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 2818–2826, doi: [10.1109/CVPR.2016.308](https://doi.org/10.1109/CVPR.2016.308).
- [20] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017, doi: [10.1109/TPAMI.2016.2577031](https://doi.org/10.1109/TPAMI.2016.2577031).
- [21] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 386–397, Feb. 2020, doi: [10.1109/TPAMI.2018.2844175](https://doi.org/10.1109/TPAMI.2018.2844175).
- [22] J. Dai, Y. Li, K. He, and J. Sun, "R-FCN: Object detection via region-based fully convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 379–387.
- [23] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot MultiBox detector," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Amsterdam, Netherlands, Oct. 2016, pp. 21–37, doi: [10.1007/978-3-319-46448-0_2](https://doi.org/10.1007/978-3-319-46448-0_2).
- [24] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 779–788, doi: [10.1109/CVPR.2016.91](https://doi.org/10.1109/CVPR.2016.91).
- [25] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 6517–6525, doi: [10.1109/CVPR.2017.690](https://doi.org/10.1109/CVPR.2017.690).
- [26] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [27] C.-Y. Wang, A. Bochkovskiy, and H.-Y. Mark Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," 2022, *arXiv:2207.02696*.
- [28] G. Jocher, A. Chaurasia, and J. Qiu. (Jan. 2023). *YOLO By Ultralytics*. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [29] C. Chen, C. Wang, B. Liu, C. He, L. Cong, and S. Wan, "Edge intelligence empowered vehicle detection and image segmentation for autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 13023–13034, Nov. 2023, doi: [10.1109/TITS.2022.3232153](https://doi.org/10.1109/TITS.2022.3232153).
- [30] S. Liang, H. Wu, L. Zhen, Q. Hua, S. Garg, G. Kaddoum, M. M. Hassan, and K. Yu, "Edge YOLO: Real-time intelligent object detection system based on edge-cloud cooperation in autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 25345–25360, Dec. 2022, doi: [10.1109/TITS.2022.3158253](https://doi.org/10.1109/TITS.2022.3158253).
- [31] Y. Mamani-Coaquira, S. L. Palacios Salcedo, Y. Guzman-Monteza, and E. Alvarez, "Detection of vehicle density based on YOLOV7 with images from Cusco, Peru," in *Proc. 3rd Int. Conf. Electr., Comput., Commun. Mechatronics Eng. (ICECCME)*, Tenerife, Spain, Jul. 2023, pp. 1–6, doi: [10.1109/iceccme57830.2023.10253099](https://doi.org/10.1109/iceccme57830.2023.10253099).
- [32] P. Kalgaonkar and M. El-Sharkawy, "NextDet: Efficient sparse-to-dense object detection with attentive feature aggregation," *Future Internet*, vol. 14, no. 12, p. 355, Nov. 2022, doi: [10.3390/fi14120355](https://doi.org/10.3390/fi14120355).
- [33] S. Li, J. Chen, W. Peng, X. Shi, and W. Bu, "A vehicle detection method based on disparity segmentation," *Multimedia Tools Appl.*, vol. 82, no. 13, pp. 19643–19655, May 2023, doi: [10.1007/s11042-023-14360-x](https://doi.org/10.1007/s11042-023-14360-x).
- [34] M. S. Junayed, M. B. Islam, A. Sadeghzadeh, and T. Aydin, "Real-time YOLO-based heterogeneous front vehicles detection," in *Proc. Int. Conf. Innov. Intell. Syst. Appl. (INISTA)*, Kocaeli, Turkey, Aug. 2021, pp. 1–7, doi: [10.1109/INISTA52262.2021.9548650](https://doi.org/10.1109/INISTA52262.2021.9548650).
- [35] U. Mittal, P. Chawla, and R. Tiwari, "EnsembleNet: A hybrid approach for vehicle detection and estimation of traffic density based on faster R-CNN and YOLO models," *Neural Comput. Appl.*, vol. 35, no. 6, pp. 4755–4774, Feb. 2023, doi: [10.1007/s00521-022-07940-9](https://doi.org/10.1007/s00521-022-07940-9).
- [36] X. Jia, Y. Tong, H. Qiao, M. Li, J. Tong, and B. Liang, "Fast and accurate object detector for autonomous driving based on improved YOLOv5," *Sci. Rep.*, vol. 13, no. 1, p. 9711, Jun. 2023, doi: [10.1038/s41598-023-36868-w](https://doi.org/10.1038/s41598-023-36868-w).
- [37] P. Deshmukh, S. Majhi, U. K. Sahoo, and S. K. Das, "Vehicle detection in diverse traffic using an ensemble convolutional neural backbone via feature concatenation," *Transp. Lett.*, vol. 16, no. 8, pp. 838–856, Aug. 2023, doi: [10.1080/19427867.2023.2250622](https://doi.org/10.1080/19427867.2023.2250622).
- [38] S. P. Krishnendhu and P. Mohandas, "DETR-SPP: A fine-tuned vehicle detection with transformer," *Multimedia Tools Appl.*, vol. 83, no. 9, pp. 25573–25594, Aug. 2023, doi: [10.1007/s11042-023-16502-7](https://doi.org/10.1007/s11042-023-16502-7).
- [39] T. R. Toha, M. Rahaman, S. I. Salim, M. Hossain, A. M. Sadri, and A. B. M. A. Al Islam, "DhakaNet: Unstructured vehicle detection using limited computational resources," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Auckland, New Zealand, Dec. 2021, pp. 1367–1372, doi: [10.1109/ICDM51629.2021.00172](https://doi.org/10.1109/ICDM51629.2021.00172).
- [40] M. Ashkanani, A. AlAjmi, A. Alhayan, Z. Esmail, M. AlBedaiwi, and M. Nadeem, "A self-adaptive traffic signal system integrating real-time vehicle detection and license plate recognition for enhanced traffic management," *Inventions*, vol. 10, no. 1, p. 14, Feb. 2025, doi: [10.3390/inventions10010014](https://doi.org/10.3390/inventions10010014).
- [41] X. Liang, X. Du, G. Wang, and Z. Han, "A deep reinforcement learning network for traffic light cycle control," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1243–1253, Feb. 2019, doi: [10.1109/TVT.2018.2890726](https://doi.org/10.1109/TVT.2018.2890726).
- [42] S. Park, E. Han, S. Park, H. Jeong, and I. Yun, "Deep Q-network-based traffic signal control models," *PLoS ONE*, vol. 16, no. 9, Sep. 2021, Art. no. e0256405, doi: [10.1371/journal.pone.0256405](https://doi.org/10.1371/journal.pone.0256405).
- [43] J. Shang, S. Meng, J. Hou, X. Zhao, X. Zhou, R. Jiang, L. Qi, and Q. Li, "Graph-based cooperation multiagent reinforcement learning for intelligent traffic signal control," *IEEE Internet Things J.*, vol. 12, no. 10, pp. 14362–14374, May 2025, doi: [10.1109/JIOT.2025.3525640](https://doi.org/10.1109/JIOT.2025.3525640).
- [44] S. Bouktif, A. Cheniki, A. Ouni, and H. El-Sayed, "Deep reinforcement learning for traffic signal control with consistent state and reward design approach," *Knowl.-Based Syst.*, vol. 267, May 2023, Art. no. 110440, doi: [10.1016/j.knsys.2023.110440](https://doi.org/10.1016/j.knsys.2023.110440).
- [45] X. Liang, X. Du, G. Wang, and Z. Han, "Deep reinforcement learning for traffic light control in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1243–1253, Feb. 2019.
- [46] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal, and T. Hester, "Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis," *Mach. Learn.*, vol. 110, no. 9, pp. 2419–2468, Sep. 2021, doi: [10.1007/s10994-021-05961-4](https://doi.org/10.1007/s10994-021-05961-4).
- [47] A. Dodia, S. Kumar, R. Rani, S. K. Pippal, and P. Meduri, "EVATL: A novel framework for emergency vehicle communication with adaptive traffic lights for smart cities," *IET Smart Cities*, vol. 5, no. 4, pp. 254–268, Dec. 2023, doi: [10.1049/smc2.12068](https://doi.org/10.1049/smc2.12068).
- [48] V. M. Madrigal Arteaga, J. R. Pérez Cruz, A. Hurtado-Beltrán, and J. Trumpold, "Efficient intersection management based on an adaptive fuzzy-logic traffic signal," *Appl. Sci.*, vol. 12, no. 12, p. 6024, Jun. 2022, doi: [10.3390/app12126024](https://doi.org/10.3390/app12126024).
- [49] M. A. Khan, F. A. Dharejo, F. Deebe, S. Ashraf, J. Kim, and H. Kim, "Toward developing tangling noise removal and blind inpainting mechanism based on total variation in image processing," *Electron. Lett.*, vol. 57, no. 11, pp. 436–438, May 2021, doi: [10.1049/ell2.12148](https://doi.org/10.1049/ell2.12148).
- [50] C. Agnew, A. Scanlan, P. Denny, E. M. Grua, P. van de Ven, and C. Eising, "Annotation quality versus quantity for object detection and instance segmentation," *IEEE Access*, vol. 12, pp. 140958–140977, 2024, doi: [10.1109/ACCESS.2024.3467008](https://doi.org/10.1109/ACCESS.2024.3467008).
- [51] J.-C. Klie, R. E. D. Castilho, and I. Gurevych, "Analyzing dataset annotation quality management in the wild," *Comput. Linguistics*, vol. 50, no. 3, pp. 817–866, Sep. 2024, doi: [10.1162/coli_a_00516](https://doi.org/10.1162/coli_a_00516).
- [52] Roboflow. *Roboflow: Annotate, Manage, and Train Computer Vision Models on Image Data*. [Online]. Available: <https://roboflow.com>
- [53] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, Sep. 2013, doi: [10.1177/0278364913491297](https://doi.org/10.1177/0278364913491297).
- [54] M. A. Kenk and M. Hassaballah, "DAWN: Vehicle detection in adverse weather nature," *IEEE Dataport*, Jul. 2020, doi: [10.21227/bw1x-yh39](https://doi.org/10.21227/bw1x-yh39).

- [55] NVIDIA. (2023). *TensorRT Developer Guide*. Accessed: Aug. 1, 2025. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/architecture-overview.html>
- [56] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "BDD100K: A diverse driving dataset for heterogeneous multitask learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, Jun. 2020, pp. 2633–2642, doi: [10.1109/CVPR42600.2020.00271](https://doi.org/10.1109/CVPR42600.2020.00271).
- [57] G. Varma, A. Subramanian, A. Nambodiri, M. Chandraker, and C. V. Jawahar, "IDD: A dataset for exploring problems of autonomous navigation in unconstrained environments," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Waikoloa Village, HI, USA, Jan. 2019, pp. 1743–1751, doi: [10.1109/WACV.2019.00190](https://doi.org/10.1109/WACV.2019.00190).
- [58] A. Ammar, A. Koubaa, M. Ahmed, A. Saad, and B. Benjdira, "Vehicle detection from aerial images using deep learning: A comparative study," *Electronics*, vol. 10, no. 7, p. 820, Mar. 2021, doi: [10.3390/electronics10070820](https://doi.org/10.3390/electronics10070820).
- [59] PSU Car Dataset. Accessed: Aug. 27, 2025. [Online]. Available: <https://github.com/aniskoubaa/psu-1810-car-dataset>
- [60] M. A. Hedeya, A. H. Eid, and R. F. Abdel-Kader, "A super-learner ensemble of deep networks for vehicle-type classification," *IEEE Access*, vol. 8, pp. 98266–98280, 2020, doi: [10.1109/ACCESS.2020.2997286](https://doi.org/10.1109/ACCESS.2020.2997286).
- [61] L. Yang, P. Luo, C. C. Loy, and X. Tang, "A large-scale car dataset for fine-grained categorization and verification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Boston, MA, USA, Jun. 2015, pp. 3973–3981, doi: [10.1109/CVPR.2015.7299023](https://doi.org/10.1109/CVPR.2015.7299023).
- [62] M. Kenk. DAWN. Accessed: Aug. 27, 2025. [Online]. Available: <https://data.mendeley.com/datasets/1819dawn>
- [63] T. Kumar, R. Brennan, A. Mileo, and M. Bendecheache, "Image data augmentation approaches: A comprehensive survey and future directions," *IEEE Access*, vol. 12, pp. 187536–187571, 2024, doi: [10.1109/ACCESS.2024.3470122](https://doi.org/10.1109/ACCESS.2024.3470122).
- [64] A. Mumuni and F. Mumuni, "Data augmentation: A comprehensive survey of modern approaches," *Array*, vol. 16, Dec. 2022, Art. no. 100258, doi: [10.1016/j.array.2022.100258](https://doi.org/10.1016/j.array.2022.100258).
- [65] S. Wu, X. Gan, J. Xiao, C. Ma, T. Deng, Z. Du, and W. Qiu, "Online insulator defects detection and application based on YOLOv7-tiny algorithm," *Frontiers Energy Res.*, vol. 12, Mar. 2024, Art. no. 1372618, doi: [10.3389/fenrg.2024.1372618](https://doi.org/10.3389/fenrg.2024.1372618).
- [66] G. Jocher, A. Chaurasia, and J. Qiu. (2023). *Ultralytics YOLOv8*. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [67] (2023). *YOLO-NAS By Deci Achieves State-of-the-Art Performance on Object Detection Using Neural Architecture Search*. [Online]. Available: <https://dec.ai/blog/yolo-nas-object-detection-foundation-model/>
- [68] Y. Ali, E. Awwad, M. Al-Razgan, and A. Maarouf, "Hyperparameter search for machine learning algorithms for optimizing the computational complexity," *Processes*, vol. 11, no. 2, p. 349, Jan. 2023, doi: [10.3390/pr11020349](https://doi.org/10.3390/pr11020349).
- [69] L. Zhu, L. Yu, and L. Zou, "Research on optimizing low-saturation intersection signals with consideration for both efficiency and fairness," *Appl. Sci.*, vol. 14, no. 17, p. 8047, Sep. 2024, doi: [10.3390/app14178047](https://doi.org/10.3390/app14178047).
- [70] S. Agha, S. Nazir, M. Kaleem, F. Najeeb, and R. Talat, "Performance evaluation of reduced complexity deep neural networks," *PLoS ONE*, vol. 20, no. 3, Mar. 2025, Art. no. e0319859, doi: [10.1371/journal.pone.0319859](https://doi.org/10.1371/journal.pone.0319859).
- [71] M. Kitani and H. Murakami, "One-sample location test based on the sign and Wilcoxon signed-rank tests," *J. Stat. Comput. Simul.*, vol. 92, no. 3, pp. 610–622, Feb. 2022, doi: [10.1080/00949655.2021.1968399](https://doi.org/10.1080/00949655.2021.1968399).
- [72] A. Wegener, M. Piórkowski, M. Raya, H. Hellbrück, S. Fischer, and J.-P. Hubaux, "TraCI: An interface for coupling road traffic and network simulators," in *Proc. 11th Commun. Netw. Simul. Symp. Canada*: ACM, Apr. 2008, pp. 155–163, doi: [10.1145/1400713.1400740](https://doi.org/10.1145/1400713.1400740).
- [73] A. T. Chronopoulos and C. M. Johnston, "A real-time traffic simulation system," *IEEE Trans. Veh. Technol.*, vol. 47, no. 1, pp. 321–331, May 1998, doi: [10.1109/25.661057](https://doi.org/10.1109/25.661057).
- [74] N. Singh, P. Saini, O. Shubham, R. Awasthi, A. Bharti, and N. Kumar, "Improved YOLOv5l for vehicle detection: An application to estimating traffic density and identifying over speeding vehicles on highway scenes," *Multimedia Tools Appl.*, vol. 83, no. 2, pp. 5277–5307, Jan. 2024, doi: [10.1007/s11042-023-15520-9](https://doi.org/10.1007/s11042-023-15520-9).
- [75] W. Liu, B. Zhao, Y. Zhu, T. Deng, and F. Yan, "Improving vehicle detection accuracy in complex traffic scenes through context attention and multi-scale feature fusion module," *Int. J. Speech Technol.*, vol. 55, no. 6, p. 389, Apr. 2025, doi: [10.1007/s10489-024-06146-x](https://doi.org/10.1007/s10489-024-06146-x).
- [76] A. Vellaidurai and M. Rathinam, "A novel OYOLOV5 model for vehicle detection and classification in adverse weather conditions," *Multimedia Tools Appl.*, vol. 83, no. 8, pp. 25037–25054, Aug. 2023, doi: [10.1007/s11042-023-16450-2](https://doi.org/10.1007/s11042-023-16450-2).
- [77] A. Farid, F. Hussain, K. Khan, M. Shahzad, U. Khan, and Z. Mahmood, "A fast and accurate real-time vehicle detection method using deep learning for unconstrained environments," *Appl. Sci.*, vol. 13, no. 5, p. 3059, Feb. 2023, doi: [10.3390/app13053059](https://doi.org/10.3390/app13053059).
- [78] T. Chen, P. Shirke, B. Chakravarthi, A. Vaghela, L. Da, D. Lu, Y. Yang, and H. Wei, "SynTraC: A synthetic dataset for traffic signal control from traffic monitoring cameras," 2024, *arXiv:2408.09588*.
- [79] T. Chu, J. Wang, L. Codeca, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 1086–1095, Mar. 2020, doi: [10.1109/TITS.2019.2901791](https://doi.org/10.1109/TITS.2019.2901791).
- [80] A. Saadi, N. Abghour, Z. Chiba, K. Moussaid, and S. Ali, "A survey of reinforcement and deep reinforcement learning for coordination in intelligent traffic light control," *J. Big Data*, vol. 12, no. 1, p. 84, Apr. 2025, doi: [10.1186/s40537-025-01104-x](https://doi.org/10.1186/s40537-025-01104-x).
- [81] H. Zhao, C. Dong, J. Cao, and Q. Chen, "A survey on deep reinforcement learning approaches for traffic signal control," *Eng. Appl. Artif. Intell.*, vol. 133, Jul. 2024, Art. no. 108100, doi: [10.1016/j.engappai.2024.108100](https://doi.org/10.1016/j.engappai.2024.108100).
- [82] F. Xiao, J. Lu, L. Li, W. Tu, and C. Li, "Advances in reinforcement learning for traffic signal control: A review of recent progress," *Intell. Transp. Infrastruct.*, vol. 4, p. 009, Jan. 2025, doi: [10.1093/iti/liaf009](https://doi.org/10.1093/iti/liaf009).
- [83] P. Michailidis, I. Michailidis, C. R. Lazaridis, and E. Kosmatopoulos, "Traffic signal control via reinforcement learning: A review on applications and innovations," *Infrastructures*, vol. 10, no. 5, p. 114, May 2025, doi: [10.3390/infrastructures10050114](https://doi.org/10.3390/infrastructures10050114).
- [84] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, Jun. 2010, doi: [10.1007/s11263-009-0275-4](https://doi.org/10.1007/s11263-009-0275-4).
- [85] M. Raza, S. Javed, M. Kazmi, A. Aziz, M. F. Ul Haque, and S. A. Qazi, "Approximate computing: Hardware and software techniques, tools and their applications," *J. Circuits, Syst. Comput.*, vol. 33, no. 4, Sep. 2023, Art. no. 2430001, doi: [10.1142/s0218126624300010](https://doi.org/10.1142/s0218126624300010).
- [86] X. Hao, S. Yang, R. Liu, Z. Feng, T. Peng, and B. Huang, "VSLM: Virtual signal large model for few-shot wideband signal detection and recognition," *IEEE Trans. Wireless Commun.*, vol. 24, no. 2, pp. 909–925, Feb. 2025, doi: [10.1109/TWC.2024.3496813](https://doi.org/10.1109/TWC.2024.3496813).
- [87] X. Hao, Z. Feng, T. Peng, and S. Yang, "Meta-learning guided label noise distillation for robust signal modulation classification," *IEEE Internet Things J.*, vol. 12, no. 1, pp. 402–418, Jan. 2025, doi: [10.1109/JIOT.2024.3462544](https://doi.org/10.1109/JIOT.2024.3462544).
- [88] X. Hao, S. Yang, R. Liu, Z. Feng, T. Peng, and B. Huang, "SMTCL: Continuous learning via selective multi-task coordination for adaptive signal classification," *IEEE Trans. Cognit. Commun. Netw.*, vol. 11, no. 3, pp. 1664–1681, Jun. 2025, doi: [10.1109/TCCN.2024.3485083](https://doi.org/10.1109/TCCN.2024.3485083).



MEHWISH RAZA received the Bachelor of Engineering (B.E.) degree in computer engineering and the Master of Engineering (M.E.) degree in computer architectures and system designs from the NED University of Engineering and Technology, Karachi, Pakistan, in 2012 and 2015, respectively. She is currently pursuing the Ph.D. degree in computer engineering, with a focus on the energy-efficient implementation of deep neural networks on FPGAs. She is a Lecturer with the Department of Computer and Information Systems Engineering, NED University of Engineering and Technology. Her research interests include approximate computing, edge computing, vision sensor nodes, and FPGA-based design solutions.



MAJIDA KAZMI has extensive experience in the field of artificial intelligence and embedded computer vision. She is currently an Associate Professor with the NED University of Engineering and Technology, Karachi, Pakistan. She also holds key research positions, including the Co-Principal Investigator with the Neurocomputation Laboratory under the National Centre of Artificial Intelligence, the Head of the Embedded Systems and Computer Vision Laboratory, and the Founding

Director of the Asma M. Hashmi STEM Centre. Her contributions to AI and technology have earned her multiple prestigious awards, including the National Engineering Excellence Gold Medal, in 2025, the House of Habib Ali Suleman Habib 2nd and 3rd Engineering Excellence Awards, in 2023 and 2024, the National Engineering Excellence Award, in 2024, and the FPCCI Talent of Pakistan 2024 underscoring her dedication to innovation and educational leadership.



HAMZA MUNIR KIDWAI received the bachelor's degree in computer information systems engineering from the NED University of Engineering and Technology (NEDUET), Karachi, Pakistan, in 2023. He is currently the Research Project Manager with NCL, NEDUET. His research interests include artificial intelligence, computer vision, natural language processing, edge computing, embedded systems, networking, and cybersecurity. He is particularly focused on the

development of intelligent, real-time systems for applications in language areas, healthcare, environmental monitoring, and digital security.



HASHIM RAZA KHAN received the B.E. degree in electrical engineering from the NED University of Engineering and Technology, Karachi, in 2002, the M.Sc. degree in communications engineering from RWTH Aachen, Germany, in 2006, and the Ph.D. degree in electronic engineering from the NED University of Engineering and Technology, in 2014. He was involved in research with Agilent Technologies, Germany, Infineon Technologies, and Linköping University, Sweden. He is currently

an Associate Professor with the NED University of Engineering and Technology, where he is also responsible for the Instrumentation Centre and the RF Laboratory, Electronics Design Centre, and the Neurocomputation Laboratory, National Centre of Artificial Intelligence. His research interests include circuits and system design for wide range of applications, including AI, robotics, multi-standard transceivers, and power.



SAAD AHMED QAZI received the B.E. degree in electrical engineering from NED University, Karachi, in 2001, the M.S. degree in digital signal processing applications from Lancaster University, U.K., in 2002, and the Ph.D. degree from Brunel University of London, U.K., in 2006. He is currently with NED University as a Meritorious Professor and the Dean of the Faculty of Electrical and Computer Engineering. He is also a Principal Investigator with the Neurocomputation Laboratory, National Centre of Artificial Intelligence. He has several international publications in major areas of technology. He is also working on several national and international research projects. His research interests include digital signal processing, joint time frequency analysis, data analytics, and decision support systems.



KAMRAN ARSHAD (Senior Member, IEEE) currently holds the distinguished position of the Dean of the Research and Graduate Studies and a Professor of electrical engineering with Ajman University, United Arab Emirates. He has played a pivotal role in leading numerous locally and internationally funded research projects that encompass the fields of machine learning, cognitive radio, LTE/LTE-advanced, 5G, optimization, and cognitive machine-to-machine communications.

He has made significant contributions to several European and international large-scale research projects and has over 150 technical peer-reviewed papers published in esteemed journals and international conferences. He was a recipient of three Best Paper Awards and one Best Research and Development Track Award and has chaired technical sessions in several leading international conferences. He holds the position of an editor and an associate editor of several international journals.



KHALED ASSALEH (Senior Member, IEEE) received the B.Sc. degree in electrical engineering from The University of Jordan, in 1988, the master's degree in electronic engineering from Monmouth University, West Long Branch, NJ, USA, in 1990, and the Ph.D. degree in electrical engineering from Rutgers, The State University of New Jersey, in 1993. From 2002 to 2017, he was with American University of Sharjah (AUS), as a Professor of electrical engineering, and the Vice

Provost of the Research and Graduate Studies. Prior to joining AUS, he had a nine-years research and development career in Telecom Industry in USA with Rutgers, Motorola, and Rockwell/Skyworks. He is currently the Vice Chancellor of the Academic Affairs and a Professor of electrical engineering with Ajman University. He holds 11 U.S. patents and has published over 150 articles in signal/image processing and pattern recognition and their applications. His research interests include bio-signal processing, biometrics, speech and image processing, and AI and machine learning. He served on organization committees of several international conferences, including ICIP, ISSPA, ICCSPA, MECBME, and ISMA. He also served as the guest editor for several special issues of journals.

...